

Hochschule Esslingen
Faculty of Machines and Systems

Bachelor Thesis

in Automation Technology and Product Informatics

**LLM-Orchestrated Event Matching for
Automotive Ramp-Up: Architecture and
Robustness on a Unified SOP Pipeline**

An industry-grade data & LLM architecture on Palantir Foundry

on the subject of:

*LLM-augmented early warning and root-cause analysis on a unified
SOP-centric events layer for supplier quality in automotive ramp-up*

by

Eduardo de la Rosa Flores

Student ID: 780660

Examiner: Prof. Matthias Deckert

Industry partner: BMW Group

Munich, January 2026

Declaration of Authorship

I hereby declare that I have written this thesis independently and that I have not used any sources or resources other than those indicated. All passages taken from other works are properly cited and referenced.

Where this work is based on data, systems, or materials provided by BMW Group, I have respected all applicable confidentiality, data protection, and compliance requirements. Any proprietary information has been handled according to the respective guidelines.

Place, Date

Munich, January 2026

Signature

Eduardo de la Rosa Flores

Acknowledgments

I would like to express my sincere gratitude to everyone who supported this work.

First, I thank my academic supervisor, **Prof. Matthias Deckert**, for the guidance, constructive feedback, and encouragement throughout this thesis. I am equally grateful to my industry supervisors at **BMW Group**, Alexej Zlavaski and Manuel Bischoff, for the practical insights, data access, and continuous support.

I thank the Supply Chain Quality Management team at BMW for their collaboration and for the many discussions that shaped the problem definition and the system architecture.

Munich, January 2026

Eduardo de la Rosa Flores

Abstract

Automotive ramp-up concentrates quality risk around Start of Production (SOP). As volume increases and process variation tightens, issues that were sporadic in pre-series can surface at scale, driving containment effort, scrap, and downstream impact. Relevant quality signals are scattered across systems and time references, which delays action once incidents appear post-SOP and makes it hard to see whether a problem was already “visible” earlier.

This thesis presents a practical, LLM-augmented solution: (i) a canonical, SOP-centric events layer on Palantir Foundry that unifies different incident types with stable identifiers and provenance; and (ii) a single-shot decision flow in which an internal LLM receives each post-SOP incident together with up to 50 most-recent, filtered pre-SOP candidates (compact, whitelisted context), and returns strict-schema JSON: **KNOWN/NEW**, a confidence score, a brief reason, and cited precursors. A light post-processing step enforces consistency and short-circuits empty candidate sets.

We benchmark against *Rules* and *Lexical* baselines built on the same candidate pools, and report **accuracy** and precision/recall/F1 for the NEW/KNOWN decision, together with token cost per incident and total experiment cost. On the evaluated snapshot, the single-shot LLM improves classification over baselines and produces audit-ready explanations that link post-SOP incidents to plausible pre-SOP evidence. The approach turns fragmented signals into earlier, reproducible warnings that integrate with existing Quality Management Team workflows.

Keywords: automotive ramp-up (Anlauf); SOP; early warning; supplier quality; canonical events; single-shot prompting; benchmarking; token cost; classification.

Contents

Abstract

1	Introduction	1
1.1	Problem and Motivation	1
1.2	Thesis Objective	1
1.3	Gap and Challenge	1
1.4	Contributions	2
1.5	Research Questions	3
1.6	Scope and Assumptions	3
1.7	Method Overview	4
1.8	Thesis Structure	4
2	Background and State of the Art	5
2.1	Ramp-up and Early-Warning in Manufacturing	5
2.2	SOP-Centric Event Modeling and Canonicalization	5
2.3	Matching Baselines (Related Work and Setup)	8
2.4	LLM Prompting in This Work	9
2.5	Synthesis and Implications	9
3	Industrial Context and Data	10
3.1	Automotive Ramp-up Context and Stakeholders	10
3.2	Data Sources Overview	10
3.3	Canonical Schema (Implemented)	11
3.4	Identifiers and Keys	12
3.5	SOP Alignment and Splits	12
3.6	Source-specific Context JSONs (including 8D)	13
3.7	Coverage, Quality, and Bias	14
3.8	Privacy, Compliance, and Aggregation	14
3.9	Summary	14

CONTENTS

4	System Architecture and Orchestration	15
4.1	Overview	15
4.2	Dataflow and Modules	16
4.3	Single-shot Orchestration	17
4.4	Prompt Contract and Validation	19
4.5	Post-processing	19
4.6	Baselines Implemented	20
4.7	Configuration and Versioning	20
4.8	Monitoring, Lineage, and Reproducibility	20
4.9	Produced Datasets	21
4.10	Summary	21
5	Methodology	22
5.1	Problem Formulation	22
5.2	Inputs and Canonical Views	22
5.3	Candidate Retrieval	23
5.4	Single-shot Prompting and Output Contract	23
5.5	Validation and Guardrails	24
5.6	Post-processing (A3)	24
5.7	Baselines	25
5.8	Outputs and Persistence	26
5.9	Method Summary	26
6	Implementation	27
6.1	Environment and Repositories	27
6.2	Per-Source Normalization	28
6.3	Canonical Union and SOP Alignment	28
6.4	Candidate Retrieval (Capped Top-50)	29
6.5	LLM Orchestration (Single-shot)	29
6.6	Evaluation and Metrics Transforms	30
6.7	Dashboards for Visualization	31
6.8	Reproducibility and Lineage	32
7	Experimental Evaluation	34
7.1	Goals	34
7.2	Evaluation Data	34
7.3	Systems Under Test	35
7.4	Metrics	35
7.5	Protocol	36
7.6	Main Results	36

CONTENTS

7.7	Candidate Set Size	37
7.8	Confidence Calibration	38
7.9	Cost Accounting	38
7.10	Qualitative Examples	39
7.11	Limitations	39
7.12	Summary	40
8	Results	41
8.1	Main Quantitative Results	41
8.2	Per-Source Breakdown	42
8.3	Behaviour by Candidate Set Size	42
8.4	Calibration of Confidence	43
8.5	Operational Cost and Scalability	44
8.6	Qualitative Examples	44
8.7	Timeline and Dashboard View	45
8.8	Error Patterns and Limitations	46
8.9	Summary	46
9	Discussion	47
9.1	Interpretation of Results	47
9.2	Design Choices and Their Effects	48
9.3	Operational Implications	48
9.4	Error Patterns and Mitigations	49
9.5	Limitations and Threats to Validity	50
9.6	Generalization and Reuse	51
9.7	Practitioner Guidance	51
9.8	Discussion Summary	52
10	Operational Integration and Governance	53
10.1	Stakeholders and Roles (RACI)	53
10.2	Human-in-the-Loop Workflow	53
10.3	Auditability and Lineage	54
10.4	Change Management and Versioning	54
10.5	Access, Privacy, and Compliance	55
10.6	Cost Considerations (Token Usage)	55
10.7	Maintenance Checklist	55
11	Conclusions and Future Work	56
11.1	Summary of Contributions	56
11.2	Answers to the Research Questions	57

CONTENTS

11.3 Limitations	58
11.4 Future Work	58
11.5 Concluding Remarks	59
References	60
Extended BMW Glossary	61

Chapter 1

Introduction

1.1 Problem and Motivation

Ramp-up around the Start of Production (SOP) concentrates quality risk. As volume increases and process variation tightens, issues that were sporadic in pre-series can surface at scale, driving containment effort, scrap, and downstream impact. Early signals already exist before and around SOP across heterogeneous sources, but they are fragmented across systems and time references. The lack of an SOP-centric, unified view delays actionable insight and forces reactive triage once incidents occur post-SOP.

This thesis tackles this problem by (i) building a canonical, SOP-aligned events layer and (ii) applying a *single-shot* LLM decision flow that, for each post-SOP incident, consumes a compact bundle of pre-SOP candidates and returns a strict-JSON verdict **KNOWN** vs. **NEW** with a short, auditable reason.

1.2 Thesis Objective

Design and evaluate an SOP-centric pipeline that (i) unifies heterogeneous sources into a canonical events dataset and (ii) classifies each post-SOP incident as **KNOWN** or **NEW** using a *single-shot* LLM call with compact pre-SOP context, followed by deterministic post-processing to guarantee usable outputs.

1.3 Gap and Challenge

Gap. Supplier-quality signals are not consistently aligned to SOP and are difficult to link to plausible pre-SOP precursors in a reproducible and explainable way.

Challenges.

1. Constructing a canonical events layer with conformed identifiers (part number, model range, supplier, plant) and harmonized timestamps.
2. Computing a robust SOP offset (`sop_delta_months`) to split pre- vs. post-SOP.
3. Generating per-incident candidate sets deterministically (same part; default filter `model_range`; optional `supplier_id/plant_id`) while capping the bundle to keep token budgets stable.
4. Enforcing a schema-bound, *single-shot* prompt that returns strict JSON suitable for downstream validation.
5. Implementing lightweight, auditable baselines that operate on the same candidate pool as the LLM.

1.4 Contributions

1. **Canonical SOP-centric events layer.** A reusable schema and pipeline in Palantir Foundry integrating main incidents such as Process Series (PCS), Vehicle Reworks, Powertrain Reworks, Sorting Actions, and PQM problem reports. All events share conformed identifiers and the field `sop_delta_months`, which encodes the difference between the incident date and the SOP date of the corresponding model range.
2. **Candidate pairing and JSON payload.** For every post-SOP event, a capped (max. 50), recency-ordered set of pre-SOP candidates and a compact, source-aware JSON representation of the incident context.
3. **LLM single-shot with deterministic post-processing.** One LLM call per incident (fixed system prompt plus user JSON). Post-processing rules convert unusable outputs into consistent labels and explanations; when no pre-events exist, the system short-circuits to a classification **NEW**.
4. **Implemented baselines.** A *rules* baseline (hand-crafted filters on the same candidate pool) and a *lexical* baseline (simple textual similarity) for direct comparison with the LLM.
5. **Produced datasets and cost accounting.** A pairing dataset (`events_pairs_for_llm_out`), an LLM output and cost dataset (`events_llm_call_and_costs`, including classification, confidence, reasons, matches, tokens, USD), a unified evaluation table with baselines and LLM, aggregated metrics by model, and a cost summary.

1.5 Research Questions

The overall goal of this thesis is to support ramp-up quality engineers in identifying whether a post-SOP incident was already “visible” in pre-SOP signals, and to do so in an auditable and cost-aware way. Concretely, we investigate the following five research questions:

- RQ1 (Architecture).** How does an SOP-centric canonical events layer with stable identifiers and compact, source-specific context influence the ability to detect KNOWN incidents across heterogeneous sources?
- RQ2 (LLM vs. baselines).** To what extent does a single-shot LLM classifier, combined with lightweight post-processing, outperform simple Rules and Lexical baselines on the KNOWN/NEW decision?
- RQ3 (Design choices and candidate sets).** How do key design choices—such as candidate-set construction and capping, context whitelists, and A3 post-processing—affect classification performance, calibration, and robustness?
- RQ4 (Behaviour across sources).** How does performance vary by source (PCS, IPSQ ReworkVehicle, Powertrain Reworks, SortingActions, PQM, Claims) and text regime (rich vs. terse remarks), and what does this imply for retrieval filters and context design?
- RQ5 (Operational impact and cost).** What lead time and token/USD cost characteristics does the system achieve on realistic ramp-up data, and are these compatible with day-to-day use in supplier-quality decision support?

1.6 Scope and Assumptions

The scope covers PCS, Vehicle Reworks, Powertrain Reworks, Sorting Actions, and PQM problem reports as main incident types, all mapped to multiple plants, suppliers, and parts within a selected, currently ramping model range with sufficient coverage. All records are mapped to the canonical schema and split by `sop_delta_months`. The system uses a single model via an internal LLM with fixed temperature to reduce output variability. Evaluation is performed on a single labelled snapshot; all methods are compared on the same set. Reported metrics are *accuracy*, *precision*, *recall*, *F1*, and cost. We focus on a single ramp-up model range (referred to as Model X in this thesis), for which SOP is in late 2025.

1.7 Method Overview

(1) **Canonical events.** Normalization, conformed IDs, SOP alignment, and compact per-source contexts.

(2) **Candidate set (pre-SOP).** Same `part_number`; default `model_range` filter; optional `supplier_id/plant_id`; capped context where $C \leq 50$, most recent first.

(3) **Single-shot prompt.** Fixed system prompt; user JSON with `post_event` and `pre_events`. Strict output: `classification` $\in \{\text{NEW}, \text{KNOWN}\}$, `confidence`, `reason`, `matching_pre_events`.

(4) **Post-processing.** Deterministic checks produce stable labels and explanations; if $C = 0$, output NEW.

(5) **Baselines.** *Rules* (selected filters) and *lexical* (simple similarity) over the same candidate pool.

(6) **Persistence and cost.** Per-row outputs with tokens and USD; aggregated metrics and cost summary.

1.8 Thesis Structure

Chapter 2 reviews background. Chapter 3 presents data sources and the canonical schema. Chapter 4 details the system architecture. Chapter 5 describes the method (single-shot LLM, post-processing, baselines). Chapter 6 covers the implementation on Palantir Foundry. Chapter 7 explains the evaluation setup and metrics. Chapter 8 reports results and qualitative examples. Chapter 9, Chapter 10, and Chapter 11 discuss implications, operationalization, and conclusions.

Chapter 2

Background and State of the Art

2.1 Ramp-up and Early-Warning in Manufacturing

Ramp-up (Anlauf) concentrates operational and quality risk as production moves from pilot to series volumes. Process variation tightens while throughput rises, so defects that were sporadic in pre-series can scale quickly and drive containment, rework, and downstream impact when detected late. Reviews of ramp-up management emphasize systematic risk identification and stabilization measures, where time pressure and complexity amplify the cost of late discovery [1], [2].

Early-warning approaches aim to surface *leading indicators* with enough lead time to act before incidents propagate at volume. In this thesis, SOP is the decision boundary between precursor evidence (pre-SOP) and consequences (post-SOP). Lead time is the advance between a matched pre-SOP signal and its linked post-SOP incident [1], [3].

2.2 SOP-Centric Event Modeling and Canonicalization

We define an *event* as a timestamped, source-specific quality signal linked to a part, supplier, plant, and model range. To reason consistently with respect to Start of Production (SOP) across heterogeneous systems, all sources are mapped to a canonical schema with conformed identifiers, harmonized timestamps, and explicit SOP alignment, while retaining source-level provenance for audits and explanations. This design follows best practice for repeatable analytics in operational settings [4].

2.2.1 Canonical Event Schema and Granularity

Table 2.1 summarizes the minimal fields present for every event. The schema enforces event-level granularity, stable keys (idempotency), harmonized time policy, and an SOP-relative offset for pre/post splitting. The `context` field is a compact, source-specific JSON (whitelisted fields) that preserves essential semantics while controlling token budgets for downstream matching and reasoning.

Table 2.1: Canonical event schema: fields, types, and provenance.

Field	Type	Filled from	Notes
<code>event_id</code>	string	generated	Stable, idempotent key (e.g., hash of <code>evt_source</code> + <code>source_id</code>).
<code>evt_source</code>	enum	source	One of {PCS, Vehicle Reworks, Powertrain Reworks, Sorting Actions, PQM}.
<code>source_id</code>	string	source	Original ID in the source system (provenance).
<code>event_date</code>	timestamp	source	UTC policy; date component used for pre/post-SOP split.
<code>event_week</code>	timestamp	derived	Week-truncated <code>event_date</code> for aggregation.
<code>model_range</code>	string	source/dim	Normalized code; alias handling documented.
<code>plantid</code>	string	source	Normalized plant code.
<code>supplier_id</code>	string	source	Normalized supplier identifier; alias resolution.
<code>supplier_name</code>	string	source	Normalized supplier name.
<code>part_number</code>	string	source	Normalized format (leading zeros, hyphens, casing).
<code>part_desc</code>	string	source	Short descriptor when available.
<code>sop_date</code>	date	dim	SOP of the model range.
<code>sop_delta_months</code>	int	computed	<code>months_between(event_date, sop_date)</code> ; pre-SOP ≤ 0 , post-SOP > 0 .
<code>original_row_id</code>	string	source	Lineage pointer to raw record.
<code>ingest_ts</code>	timestamp	platform	Pipeline ingest time (audit).
<code>context</code>	json	source	Compact, source-specific whitelist used for matching and explanations.

2.2.2 SOP Alignment and Pre/Post Split

Each record receives `sop_delta_months` from the event timestamp and the model-range SOP. We use this to establish a consistent decision boundary: *pre-SOP* (≤ 0) as precursor evidence versus *post-SOP* (> 0) as consequences. This alignment enables consistent lead-time computation and fair cross-source evaluation of any early-warning mechanism [1].

2.2.3 Data Quality, Provenance, and Lineage

To ensure trust and auditability we: (i) define required/not-null fields per source, (ii) validate temporal consistency against SOP windows, (iii) deduplicate by stable keys, and (iv) keep a lineage pointer (`original_row_id`) back to the raw record. This addresses common sources of technical debt in data-centric systems by making dependencies explicit and recoverable [5], [6].

2.2.4 Keys, Joins, and Source-Specific Context

Matching relies primarily on normalized `part_number`, filtered by `model_range` and optionally by `supplier_id/plantid` to avoid ambiguous merges. Figure 2.1 sketches the core entities (Part, Model Range, Supplier, Plant, Event) and the anti-ambiguity strategy. On the right, it illustrates compact `context` retained per source (examples):

- **PCS:** chapter, question text, BMW/supplier comments, rating, action status.
- **Vehicle Reworks:** defect place/type, brief remark.
- **Powertrain Reworks:** defect place/type, brief remark.
- **Sorting Actions:** campaign name, aggregated defect place, short remark.
- **PQM:** problem title, cluster, short description, causes, measures, and links to related claims.
- **8D Reporting (part-level):** title, problem cause description, corrective actions, introduced corrective actions.

The compact context keeps payloads informative but bounded, supporting both transparent baselines and LLM-based classifiers without excessive noise [4].

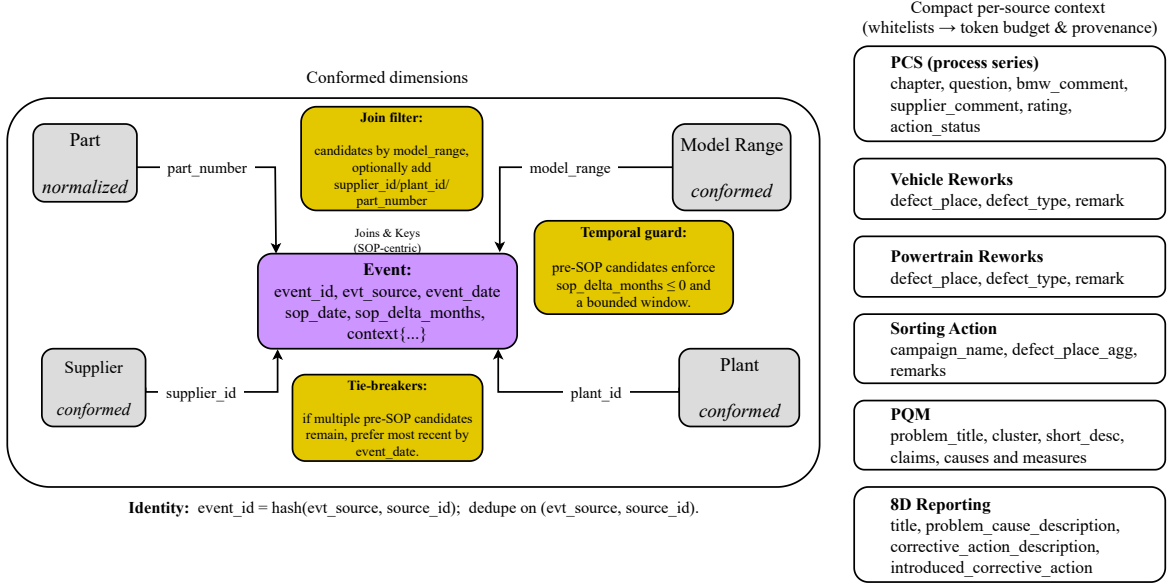


Figure 2.1: Join keys and anti-ambiguity strategy across part, model range, supplier, and plant; compact context per source and 8D Reporting at part level.

2.3 Matching Baselines (Related Work and Setup)

As transparent comparators to LLM-based matching, two families are considered, both operating on the same candidate pools (same part; default model_range filter; optional supplier/plant filters):

Rules baseline. Deterministic patterns on normalized fields and compact context, constrained to pre-SOP by $\text{sop_delta_months} \leq 0$. If any candidate satisfies the rule set, the post-SOP incident is labeled **KNOWN**; otherwise **NEW**. Specific rules and thresholds are detailed later in Chapter 5.

Lexical baseline. Short, normalized texts are built for post and pre-events; a simple lexical similarity (e.g., TF-IDF/BM25-style scoring or cosine over token vectors) determines **KNOWN** when above a threshold [7], [8]. Exact text construction and thresholds are defined in Chapter 5.

Embedding-based similarity can extend coverage to paraphrases [9], but in this work it is treated as related background; the empirical comparison focuses on rules, lexical similarity, and the LLM classifier.

2.4 LLM Prompting in This Work

The prompting strategy is schema-bound and SOP-centric. For each post-SOP incident, a *single-shot* call receives:

- a `post_event` block (canonical fields plus compact `context`),
- a capped list of `pre_events[]` from the same part key, filtered by `model_range` and optionally `supplier/plant`.

The model returns strict JSON with `classification` $\in \{\text{NEW}, \text{KNOWN}\}$, numeric `confidence`, a short `reason`, and `matching_pre_events`. Deterministic post-processing enforces usable outputs (e.g., rejecting empty match lists for a `KNOWN` label). Detailed prompt design and post-processing steps are described in Chapter 5; here we position this setup as an instance of structured, schema-bound prompting for industrial data.

2.5 Synthesis and Implications

Prior work explains why ramp-up concentrates risk and why lead-time matters [1]. A canonical, SOP-aligned events layer with conformed identifiers provides the substrate for fair, repeatable analytics across heterogeneous quality sources [4]. Rules and lexical approaches are transparent and fast [7], [8]; they serve as practical baselines against which a single-shot LLM classifier can be compared under the same SOP-centric candidate pools. The remainder of the thesis builds on this foundation: Chapter 3 introduces the industrial data and canonical schema; Chapter 4 describes the pipeline and datasets; Chapter 5 formalizes the method (candidate construction, JSON payloads, post-processing, baselines); Chapter 7 defines metrics (*accuracy*, *precision*, *recall*, *F1*) and cost reporting; and Chapter 8 presents empirical findings with qualitative examples.

Chapter 3

Industrial Context and Data

3.1 Automotive Ramp-up Context and Stakeholders

Within BMW’s organization, multiple teams monitor supplier- and plant-side quality signals around Start of Production (SOP). During ramp-up (Anlauf), scattered observations from process readiness, in-plant rework, sorting actions, process series and engineering problem reports must be consolidated into a coherent picture to support timely, auditable decisions. This chapter describes the industrial context and the data assets used in this thesis, independent of any single department or workflow specialization. The scope covers a single ramp-up model range, referred to as Model X in this thesis, relevant plants with sufficient coverage, and a multi-month window around SOP.

3.2 Data Sources Overview

We work with six operational sources that, together, capture pre-/post-SOP quality signals at different levels of granularity. Table 3.1 summarizes the sources, the unit of analysis for each record, the compact text fields used downstream, and illustrative uses in the pipeline.

Table 3.1: Supplier-quality data sources used in this thesis.

Source	Granularity	Key text fields (compact)	Example use
PCS (<i>process series</i>)	process-level	questions; BMW and supplier comments; action status	process readiness
Vehicle Reworks	vehicle-level	defect place; defect type; short remark	in-plant defects
Powertrain Reworks	vehicle-level	defect place; defect type; short remark	in-plant defects
Sorting Actions	campaign-level	campaign title; aggregated defect place; short remark	suspect-part sorting
PQM (problem reports)	part-level	problem title; cluster; short description; causes and measures	engineering problems
8D Report	part-level	problem cause description; corrective action; introduced action description	part problems and solutions

3.3 Canonical Schema (Implemented)

Following Chapter 2, all sources are normalized into an SOP-centric, canonical events schema that enforces event-level granularity, stable identifiers, harmonized timestamps, and an explicit SOP-relative offset. We reuse the canonical fields summarized in Table 2.1 (see Chapter 2, §2.2) and highlight the following implementation aspects:

- **Stable keys.** `event_id` is generated deterministically from `evt_source`, `source_id`, and normalized part/supplier identifiers; conformed keys exist for `part_number`, `model_range`, `supplier_id`, and `plantid`.
- **Time policy.** `event_date` is stored in UTC; an ISO week truncation (`event_week`) is materialized for aggregation and timeline visualizations.
- **SOP alignment.** `sop_date` comes from dimensional tables; `sop_delta_months = months_between(event_date, sop_date)` is used for the pre-/post-SOP split and for lead-time computation.
- **Context JSONs.** A compact, source-specific whitelist (Section 3.6) limits noise and preserves token budgets for candidate matching and LLM prompting.

3.4 Identifiers and Keys

To avoid ambiguous merges across heterogeneous systems, identifiers are normalized and conformed as follows:

- **Part.** `part_number` normalization handles leading zeros, hyphens, casing, and common formatting aliases. Where available, a conformed material identifier is used consistently across sources.
- **Model range.** `model_range` values are standardized to a single code system with alias handling; for parts used in multiple model ranges, a default rule (e.g. first model usage) is applied unless stated otherwise.
- **Supplier and plant.** `supplier_id` and `plantid` are conformed with alias resolution, so textual variants (names, regional codes) map to a canonical key.
- **Idempotency.** `event_id` uniquely identifies an event; duplicates are removed on (`evt_source`, `source_id`) to ensure stable joins and aggregations.

Tie-breaking rules for candidate retrieval (e.g. preferring most recent pre-SOP events when multiple candidates are equivalent) are documented later together with the matching method.

3.5 SOP Alignment and Splits

Each event receives an SOP-relative offset `sop_delta_months` computed from `event_date` and `sop_date`. We define:

- *pre-SOP*: `sop_delta_months` ≤ 0 , treated as precursor evidence,
- *post-SOP*: `sop_delta_months` > 0 , treated as consequences at series ramp-up.

This boundary underpins lead-time measurement and cross-source comparison used later in the evaluation. Figure 3.1 visualizes the split as an SOP-centered timeline.

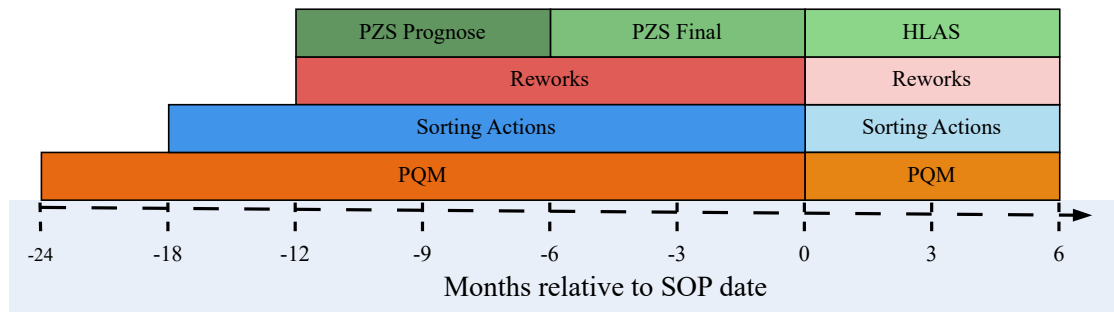


Figure 3.1: SOP-centric signal timeline. Pre-SOP window (left) with typical milestones (PCS Prognose, PCS Final) and signal types (Reworks, Sorting Actions, PQM); SOP at 0; HLAS window to +6 months. Lead time is measured between a matched pre-SOP signal and its post-SOP incident.

3.6 Source-specific Context JSONs (including 8D)

For downstream orchestration, each event carries a small, source-specific `context` JSON. Fields are whitelisted to keep the payload focused, avoid leaking identifiers that do not help matching, and preserve provenance. Where available, part-level 8D reporting data are joined in to sharpen explanations and provide actionable links to corrective actions.

Table 3.2: Compact context JSON whitelists (logical field names).

Source	Whitelisted fields (complete)
PCS	chapter; question; bmw_comment; supplier_comment; action_status
Vehicle Reworks	defect_place_name; defect_type_name; short_remark; plant_name; cluster_name_list
Powertrain Reworks	defect_place_name; defect_type_name; short_remark; plant_name; cluster_name_list
Sorting Actions	campaign_title; plant_name; defect_place_name; defect_type_name; cluster_name_list; error_description
PQM (problem reports)	problem_title; cluster_name_en; problem_description; plant_name_en_list
8D (part-level, joined to PQM)	cause_title; cause_description; cause_owner; part_numbers; measure_type; measure_title; measure_description

Note. 8D fields appear only when present in upstream systems or via joins; otherwise they remain empty. Language-specific suffixes (`_de`, `_en`) are harmonized to a single logical name when the semantics are equivalent.

3.7 Coverage, Quality, and Bias

To ensure trustworthy inputs, we summarize coverage and completeness for the canonical fields and the context JSONs (Table 3.3 shows the snapshot used in this thesis). Quality checks include required/not-null constraints on identifiers and dates, duplicate detection on stable keys, and temporal sanity checks against SOP windows. Known sources of bias include language mixing, abbreviation variance across plants, and supplier aliasing; the canonicalization layer mitigates these issues but does not eliminate them entirely.

Table 3.3: Coverage and completeness for the selected model ranges (snapshot used in this thesis).

Source	Records (N)	% with part_number	% with supplier_id
PCS	1.103	100	100
Vehicle Reworks	7.690	100	99
Powertrain Reworks	0	—	—
Sorting Actions	13.285	100	100
PQM	6.038	100	55

3.8 Privacy, Compliance, and Aggregation

The canonical layer stores only fields required for event matching, explanation, and evaluation. Context JSONs are compact and may exclude personally identifiable information or business-sensitive details; where needed, fields are aggregated or anonymized before inclusion. Access follows organizational policy, and example records in this thesis are either synthetic, anonymized, or presented at an aggregated level to avoid disclosure.

3.9 Summary

This chapter outlined the industrial context and the data assets used in the thesis. The sources span process readiness, in-plant rework, sorting actions, and engineering problem tickets, with additional part-level 8D reporting data where available. All records are unified into an SOP-centric canonical schema with conformed identifiers, harmonized timestamps, and a pre-/post-SOP split via `sop_delta_months`. Compact, source-specific context JSONs are defined with explicit field whitelists and length caps. Coverage and quality controls are in place. These foundations support the system architecture in Chapter 4 and the experimental evaluation in Chapter 7.

Chapter 4

System Architecture and Orchestration

4.1 Overview

This chapter presents the end-to-end architecture that powers the SOP-centric matching system. The pipeline ingests heterogeneous sources, normalizes them into a unified events layer, retrieves pre-SOP candidates per incident, and applies a *single-shot* LLM decision to classify each post-SOP event as **KNOWN** or **NEW** with a short, auditable reason and links to matched pre-SOP events. We focus on modularity, provenance, and reproducibility: each stage materializes explicit datasets and logs configuration for later evaluation (Chapter 7).

4.2 Dataflow and Modules

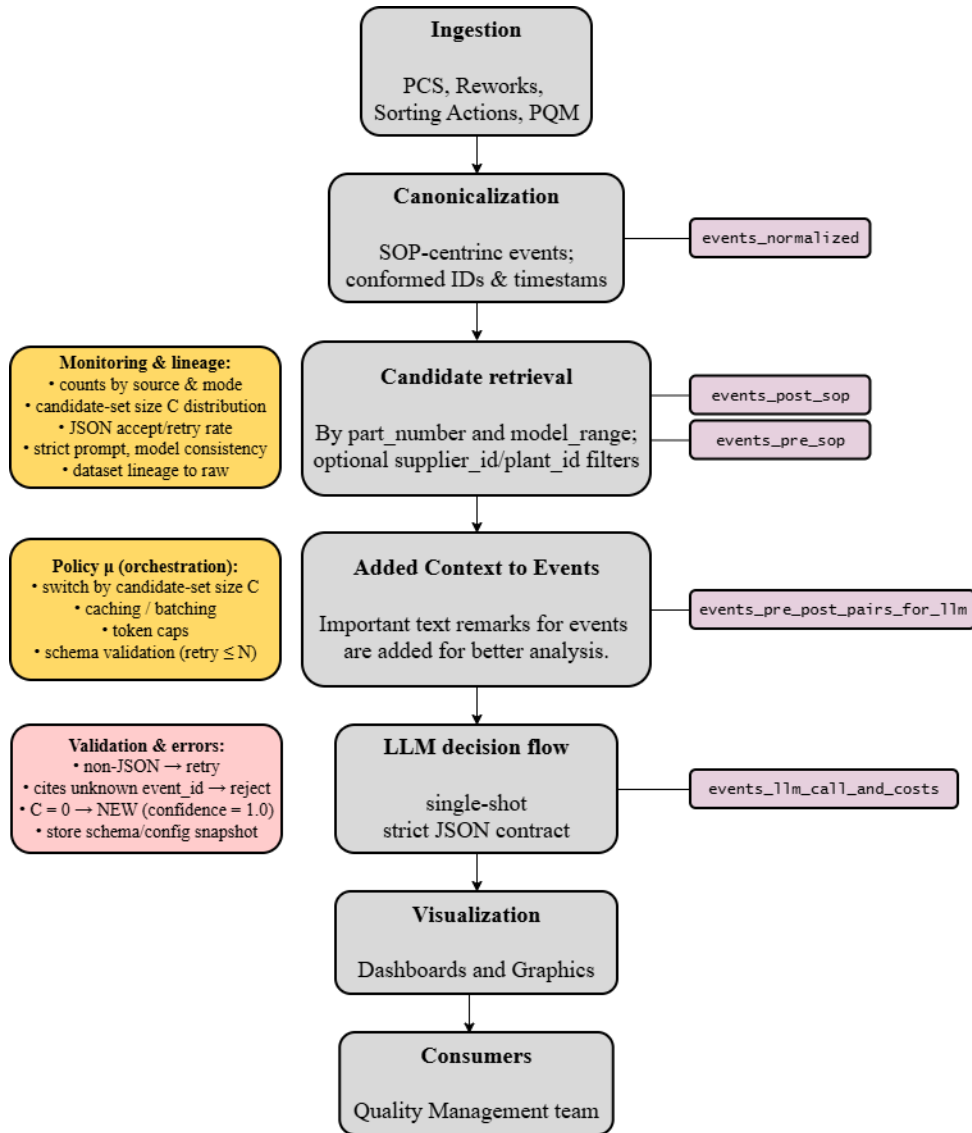


Figure 4.1: High-level dataflow: ingestion $\rightarrow$ canonicalization $\rightarrow$ candidate retrieval $\rightarrow$ single-shot LLM decision $\rightarrow$ visualization. Materialized datasets: **events_unified_out**, **events_pairs_for_llm_out**, **events_pairs_llm_out**.

4.2.1 Ingestion and Normalization

Raw source tables (PCS, Vehicle Reworks, Powertrain Reworks, Sorting Actions, PQM and related 8D data) are mapped to the canonical schema described in Chapter 3. Each normalized table keeps a pointer `original_row_id` for audit and a generated `event_id` for idempotency. Source-specific context JSONs are constructed from whitelisted text fields (see Chapter 3, Section 3.6) to keep downstream reasoning payloads compact and comparable across systems.

4.2.2 Unified SOP-Centric Events Layer

All normalized sources are unioned into a unified events dataset (referred to as `events_unified_out`) with the following key fields:

- identifiers: `evt_source`, `source_id`, `event_id`, `original_row_id`;
- time: `event_date`, `event_week`, `sop_date`, `sop_delta_months`;
- technical keys: `part_number`, `model_range`, `plantid`, `supplier_id`;
- descriptors: `plant_name_de`, `plant_name_en`, `supplier_name`, `part_desc`, `first_model_usage`;
- orchestration fields: `ingest_ts`, `part_key`, and source-specific `context`.

This dataset is the single source of truth for downstream retrieval and evaluation. The explicit `sop_delta_months` field enables the pre-/post-SOP split without re-deriving SOP logic in later stages.

4.2.3 Candidate Retrieval (Pre-SOP Set)

For each post-SOP incident, we construct a candidate set of pre-SOP events using deterministic filters on the unified layer:

- **Primary key.** Same `part_number`, implemented via a normalized `part_key`.
- **Default filter.** Same `model_range`; optional stricter filters on `supplier_id` and/or `plantid` when required to avoid ambiguous matches.
- **Temporal bound.** Only candidates with `sop_delta_months` ≤ 0 are considered (pre-SOP evidence).
- **Ordering and cap.** Candidates are sorted by `event_date` descending and limited to the most recent 50 events to keep token budgets stable.

The result is a per-incident bundle, materialized as `events_pairs_for_llm_out`, with one `post_event` and an array `pre_events[]` (optionally grouped by source in the JSON for readability).

4.3 Single-shot Orchestration

Each post-SOP incident is evaluated via a single, schema-bound prompt:

Call. One *system* message with the classification task and constraints, plus one *user* message containing the JSON payload (`post_event` and `pre_events[]`).

Model. An internal GPT-4.1-family model in the platform with fixed temperature for reduced randomness.

Return JSON. A strict JSON object with `classification` $\in \{\text{NEW}, \text{KNOWN}\}$, numeric `confidence` $\in [0, 1]$, a short `reason`, and `matching_pre_events` (list of referenced `event_ids` with per-match justifications).

Outputs and token usage are stored in `events_llm_call_and_costs`, which includes both the raw model answer and post-processed fields used in evaluation.

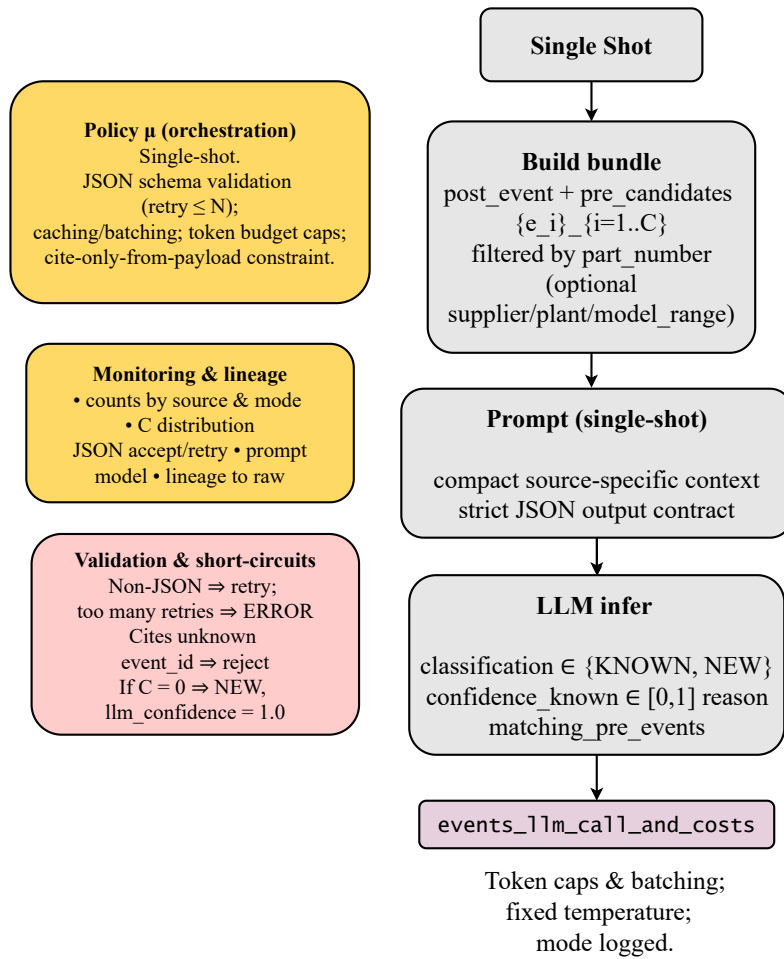


Figure 4.2: Prompt orchestration used in this work: a *single-shot* call per post-SOP incident with a compact JSON payload.

Conceptually, the orchestration can be expressed as a dispatch policy μ over different prompting modes (for example, a pure single-shot mode versus a retrieve→judge mode for very large candidate sets). In this thesis, however, μ is instantiated as a trivial policy that

always selects the single-shot mode. Designing and evaluating non-trivial, data-dependent policies (e.g. switching to retrieve→judge when the candidate count C exceeds a threshold) is left as future work.

4.4 Prompt Contract and Validation

Prompts are designed to be explicit and auditable. Each model call must return strict JSON with the following fields:

- `classification` $\in \{\text{NEW}, \text{KNOWN}\}$,
- `confidence` $\in [0, 1]$,
- `reason` (short, human-readable, referencing the context),
- `matching_pre_events` = list of objects `{event_id, justification}`.

Server-side validation enforces JSON schema and payload integrity: referenced IDs must belong to the provided `pre_events` list, and mandatory fields cannot be empty. Non-conformant outputs trigger bounded retries with the same payload and a more restrictive system message. If no valid JSON can be recovered, the incident is flagged as unusable and excluded from metric computation, but kept in the logs for analysis.

4.5 Post-processing

A lightweight, deterministic post-processing layer ensures stable labels and explanations:

- If `classification` = `KNOWN` and (`confidence` is missing or below a configured threshold, e.g. 0.6), the label is forced to `NEW` and an explanatory suffix is appended to `reason`.
- If `classification` = `KNOWN` but `matching_pre_events` is empty or refers to invalid IDs, the label is forced to `NEW` with an added explanation.
- Short-circuit rule: if a post-SOP event has no pre-SOP candidates after filtering, the model is not called. The system directly sets `classification` = `NEW`, assigns a high default `confidence`, and stores a fixed, template `reason`.

These rules guarantee that downstream consumers always see consistent labels and non-empty explanations, even when the raw model output is partially malformed.

4.6 Baselines Implemented

Two baselines are implemented on exactly the same candidate pools as the LLM:

- **Rules baseline.** Deterministic filters on normalized fields and compact `context` (e.g. matching defect place/type or cluster, constrained to pre-SOP). If at least one pre-SOP candidate satisfies the rule set, the incident is labeled `KNOWN`; otherwise `NEW`.
- **Lexical baseline.** Short, normalized texts are built for post and pre-events (using selected context fields per source). A simple lexical similarity (e.g. token cosine or Jaccard) is computed; if the maximum score in the candidate set exceeds a chosen threshold, the incident is labeled `KNOWN`, else `NEW`.

Both baselines are cheap to compute, transparent, and operate on the same SOP-centric candidate sets as the LLM, which allows direct comparison in Chapter 7.

4.7 Configuration and Versioning

To ensure reproducibility, each major run writes a compact, machine-readable configuration snapshot alongside the outputs. It includes:

- model identifier and temperature,
- candidate filters (e.g. model-range, supplier/plant options),
- candidate cap (default: 50),
- prompt template ID and JSON schema version,
- thresholds used in post-processing and baselines,
- run timestamp and data snapshot identifier.

These snapshots allow exact re-runs of the orchestration on the same data, or controlled comparisons across configurations (e.g. different thresholds or models).

4.8 Monitoring, Lineage, and Reproducibility

Every stage materializes datasets with stable schemas and lineage back to raw inputs. Logged artifacts include:

- event counts by source, plant, and model range,

- distribution of candidate set sizes C per incident,
- JSON acceptance and retry rates,
- distributions of `confidence` and class balance for `NEW` vs. `KNOWN`.

These logs feed the evaluation tables and figures in Chapter 7 and provide a basis for operational monitoring (e.g. drift in candidate set sizes or class balance over time).

4.9 Produced Datasets

Table 4.1: Key datasets produced by the architecture.

Dataset	Purpose / Key fields
<code>events_normalized</code>	Unified SOP-centric events with compact <code>context</code> .
<code>events_pairs_for_llm_out</code>	Per-incident candidate bundles (capped at 50; ordered by recency).
<code>events_llm_call_and_costs_out</code>	Per-incident LLM outputs and raw token/cost fields.
<code>events_pairs_eval_with_baselines</code>	Unified evaluation view combining Rules, Lexical and LLM decisions.
<code>events_pairs_eval_classic_metrics</code>	Classic binary metrics (accuracy, precision, recall, F1) per system.
<code>events_pairs_eval_metrics_by_src</code>	Metrics by <code>evt_source</code> (per-source breakdown).
<code>events_pairs_llm_cost_summary</code>	Aggregated token usage and cost (per source and overall).

4.10 Summary

The architecture separates concerns between canonicalization, candidate retrieval, and single-shot LLM judgment. A strict output contract and deterministic post-processing ensure auditability, while configuration snapshots and materialized lineage guarantee reproducibility. These choices enable the quantitative evaluation in Chapter 7 and the practitioner-facing views in Chapter 8.

Chapter 5

Methodology

5.1 Problem Formulation

We frame post-/pre-SOP matching as a binary decision per post-SOP incident. Let $\mathcal{E}$ denote the unified event set (Chapter 3), and define

$$\mathcal{E}^{\text{post}} = \{e \in \mathcal{E} \mid \text{sop_delta_months}(e) > 0\}, \quad \mathcal{E}^{\text{pre}} = \{e \in \mathcal{E} \mid \text{sop_delta_months}(e) \leq 0\}.$$

For each $e^{\text{post}} \in \mathcal{E}^{\text{post}}$, the system predicts

$$y \in \{\text{KNOWN}, \text{NEW}\},$$

where **KNOWN** means that at least one plausible precursor exists in $\mathcal{E}^{\text{pre}}$ for the same part (under the configured filters), and **NEW** indicates no credible pre-SOP evidence. The output must include a categorical label, a confidence score, a short explanation, and links to any matched precursors.

5.2 Inputs and Canonical Views

Each event e exposes canonical identifiers and a compact, source-specific **context** JSON (Chapter 3):

$$\begin{aligned} x(e) = (&\text{evt_source}, \text{source_id}, \text{event_id}, \text{event_date}, \text{event_week}, \\ &\text{model_range}, \text{sop_date}, \text{sop_delta_months}, \text{plantid}, \text{supplier_id}, \\ &\text{part_number}, \text{part_key}, \text{context}). \end{aligned} \tag{5.1}$$

No additional numeric feature engineering is required beyond SOP alignment and identifier normalization; the compact **context** fields provide short textual cues per source (PCS, Reworks, Sorting Actions, PQM, 8D).

5.3 Candidate Retrieval

For each post-SOP incident e^{post} , a pre-SOP candidate pool is constructed from `events_unified_out` by deterministic filters:

$$\mathcal{C}(e^{\text{post}}) = \left\{ e \in \mathcal{E}^{\text{pre}} \mid \begin{aligned} &\text{part_key}(e) = \text{part_key}(e^{\text{post}}), \\ &\text{model_range}(e) = \text{model_range}(e^{\text{post}}) \end{aligned} \right\}, \quad (5.2)$$

with optional additional restrictions on `supplier_id` and/or `plantid` in configurations where stricter matching is desired.

Candidates are sorted by `event_date` in descending order and capped at the most recent $C_{\text{max}} = 50$ events:

$$\mathcal{C}_{50}(e^{\text{post}}) = \text{Top-50 by recency from } \mathcal{C}(e^{\text{post}}).$$

When multiple events share the same timestamp, ties are broken in favour of records with richer `context`. The resulting bundles

$$(e^{\text{post}}, \{e_i\}_{i=1}^C), \quad C = |\mathcal{C}_{50}| \leq 50,$$

are materialized as `events_pairs_for_llm_out` and form the sole input to the LLM and baselines.

5.4 Single-shot Prompting and Output Contract

Each post-SOP incident is evaluated via exactly one **single-shot** LLM call using the candidate bundle above.

System message (conceptual). The system message instructs the model to decide whether the post-SOP incident is **KNOWN** (already visible in the pre-SOP candidates) or **NEW**, to rely only on the provided events, and to return strict JSON with no additional text.

User message (payload). The user message contains a compact JSON with:

- one `post_event` (canonical identifiers + source-specific `context`),
- an array `pre_events[]` (candidates from PCS, Reworks, Sorting Actions, PQM, and 8D where available).

Required return JSON. The model must return:

- `classification` $\in \{\text{NEW}, \text{KNOWN}\}$,
- `confidence` $\in [0, 1]$,
- `reason` (one concise, auditable sentence grounded in the contexts),
- `matching_pre_events` = array of objects `{event_id, justification}`.

The raw responses, together with token usage, are stored in `events_pairs_llm_out`.

5.5 Validation and Guardrails

To ensure robustness and auditability, the following checks are applied to each raw model response:

1. **JSON schema validation.** The response must parse as valid JSON and contain all required fields with admissible types. Non-conformant outputs trigger bounded retries with the same payload and stricter instructions.
2. **Payload integrity.** Every `event_id` in `matching_pre_events` must belong to the candidate set for this incident. References to unknown IDs cause the response to be rejected and retried.
3. **Determinism knobs.** A fixed temperature (set to 0.0), prompt template identifier, and model name are logged so that behaviour can be reproduced for a given configuration.
4. **Short-circuit condition.** If $C = 0$ (no candidates in $\mathcal{C}_{50}(e^{\text{post}})$), the LLM is not called. Instead, the system directly sets `classification` = `NEW`, assigns a high default `confidence`, and writes a fixed, template `reason`.

5.6 Post-processing (A3)

After validation, a lightweight, deterministic post-processing layer (A3) adjusts the model output when necessary:

- If `classification` = `KNOWN` and (`confidence` is missing or < 0.6), the label is forced to `NEW` and an explanatory suffix is appended to `reason`.
- If `classification` = `KNOWN` but `matching_pre_events` is empty or contains only invalid references, the label is forced to `NEW` and an explanatory suffix is appended.

The final decision used in all analyses is denoted *LLM+A3*. It is stored in `events_pairs_llm_out` and then joined into the evaluation table `events_pairs_eval`.

5.7 Baselines

Two lightweight baselines are computed on the same capped candidate pools $\mathcal{C}_{50}(e^{\text{post}})$, using only canonical fields and `context`.

5.7.1 Rules Baseline

The rules baseline applies hand-crafted checks over normalized fields and compact `context` within the pre-SOP window (`sop_delta_months` ≤ 0). At a high level:

- candidates must share `model_range` with e^{post} ;
- only selected `evt_source` values are considered (e.g. PCS, Vehicle Reworks, Power-train Reworks, Sorting Actions, PQM);
- simple context-level conditions are applied, such as matching defect place/type or cluster name for specific source combinations.

If at least one candidate satisfies the configured rule set, the incident is labeled `KNOWN`; otherwise `NEW`. The resulting label is stored in `events_pairs_eval` as `rules_baseline_label`.

5.7.2 Lexical Baseline

The lexical baseline reduces each event to a short text $t(e)$ assembled from its `context` fields and a minimal descriptor (e.g. part description). Texts are normalized by lowercasing, stripping whitespace, and removing trivial tokens. For a given post-SOP incident e^{post} and its candidate pool $\mathcal{C}_{50}(e^{\text{post}})$, we compute a similarity score $s(e^{\text{post}}, e)$ for each $e \in \mathcal{C}_{50}$, for example using a token-based cosine or Jaccard similarity:

$$s_{\text{Jaccard}}(e^{\text{post}}, e) = \frac{|T(e^{\text{post}}) \cap T(e)|}{|T(e^{\text{post}}) \cup T(e)|},$$

where $T(\cdot)$ denotes the set of tokens. Let

$$s_{\text{max}}(e^{\text{post}}) = \max_{e \in \mathcal{C}_{50}(e^{\text{post}})} s(e^{\text{post}}, e).$$

Given a tuned threshold τ , the lexical decision is

$$\hat{y}_{\text{lex}}(e^{\text{post}}) = \begin{cases} \text{KNOWN}, & s_{\text{max}}(e^{\text{post}}) > \tau, \\ \text{NEW}, & \text{otherwise.} \end{cases}$$

The resulting label is stored in `events_pairs_eval` as `lexical_baseline_label`.

5.8 Outputs and Persistence

The method writes decisions and auxiliary information into materialized datasets that are later consumed in Chapter 7:

- `events_pairs_for_llm_out`: per-incident bundles with post-SOP events and their pre-SOP candidate sets (up to 50, ordered by recency).
- `events_pairs_llm_out`: raw and post-processed LLM outputs (`llm_classification`, `llm_confidence`, `llm_reason`, `llm_matching_pre_events`, `llm_match_count`, `llm_postprocess` token counts and costs).
- `events_pairs_eval`: unified view joining ground-truth labels (where available), rules baseline, lexical baseline, and LLM+A3 outputs for every evaluated incident.

5.9 Method Summary

In summary, the method:

1. reduces heterogeneous quality signals into SOP-aligned events with compact, source-specific `context`;
2. constructs capped, recency-ordered pre-SOP candidate sets per post-SOP incident;
3. applies a **single-shot** LLM decision with a strict JSON contract;
4. enforces validation and deterministic A3 post-processing to guarantee usable outputs; and
5. benchmarks performance against rules and lexical baselines on identical candidate pools.

These steps provide a reproducible, SOP-centric procedure that can be evaluated quantitatively in Chapter 7 and inspected qualitatively in Chapter 8.

Chapter 6

Implementation

6.1 Environment and Repositories

The system is implemented on Palantir Foundry using PySpark transforms for data processing, Code Repositories for versioning, and Contour for visualization. All transforms are idempotent and materialize typed datasets with explicit schemas. Table 6.1 summarizes the logical layout of code and outputs.

Table 6.1: Repository and dataset structure (logical).

Path / Dataset	Purpose
01_norm/ev_{src}_norm.py	Per-source normalization to canonical columns and source-specific context JSON.
02_unified/events_unified_out.py	Union of normalized sources + SOP alignment + compact context .
03_candidates/events_pairs_for_llm_out.py	For each post-SOP incident, build capped (most recent) pre-SOP candidates (max 50).
04_llm/events_pairs_llm_out.py	Single-shot LLM call per incident; record raw outputs, post-processing flags, and token/cost fields.
05_eval/events_pairs_eval.py	Join Rules/Lexical baselines with LLM+A3 decisions for evaluation.
05_eval/events_pairs_eval_metrics.py	Aggregate metrics by system (accuracy, precision, recall, f1).
05_metrics/llm_cost_summary.py	Aggregate token usage and cost per source and overall.
dashboard/	Contour views (filters, SOP timelines, incident drill-down).

6.2 Per-Source Normalization

Each raw source has a transform `ev_{src}_norm.py` that:

1. maps raw tables to the canonical columns defined in Chapter 3 (e.g. `event_id`, `evt_source`, `source_id`, `event_date`, `model_range`, `plantid`, `supplier_id`, `part_number`, `part_desc`, `original_row_id`, `ingest_ts`);
2. enforces identifier formats (part, plant, supplier, model range) using shared helper functions and dimension tables;
3. builds a compact, source-specific `context` JSON by whitelisting the textual fields discussed in Chapter 3, including 8D fields where available.

All normalized tables keep `evt_source`, `source_id`, and `original_row_id` for lineage and de-duplication.

Identifier normalization (examples)

- **Part number.** Trim, uppercase, normalize separators, left-pad to standard length where applicable, and compute a stable `part_key`.
- **Supplier/Plant.** Map textual aliases to conformed keys via dimension tables; store the canonical key as `supplier_id` and `plantid`.
- **Model range.** Resolve synonyms to a single code and join SOP information from dimensional tables.

6.3 Canonical Union and SOP Alignment

The union transform `events_unified_out.py` vertically stacks all normalized sources into a single events table and computes SOP alignment (cf. Chapter 3):

- **Union.** Standardize column names across sources and union them with an explicit `evt_source` flag.
- **SOP enrichment.** Join a dimensional model-range table to obtain `sop_date` and compute

```
sop_delta_months = months_between(event_date, sop_date),
```

which is used for pre-/post-SOP splitting.

- **Context.** Carry the compact, source-specific `context` JSON from each normalized table without change.

The resulting dataset `events_unified_out` is the single source of truth for candidate retrieval and evaluation.

6.4 Candidate Retrieval (Capped Top-50)

Transform `events_pairs_for_llm_out.py` implements the candidate retrieval method from Chapter 5 (§5.3) on `events_unified_out`:

- select post-SOP incidents with `sop_delta_months > 0`;
- for each post-SOP event, build a pre-SOP pool with the same `part_key` and `model_range`, optionally restricting by `supplier_id` and/or `plantid`;
- filter to pre-SOP events (`sop_delta_months ≤ 0`);
- order candidates by `event_date` (descending) and keep at most 50.

The output table contains, for each post-SOP `event_id`, a compact `post_event` struct, an array `pre_events[]` with the capped candidates, and convenience fields such as `pre_events_count` and `pre_events_truncated`.

6.5 LLM Orchestration (Single-shot)

Transform `events_pairs_llm_out.py` reads `events_pairs_for_llm_out` and applies the single-shot prompting scheme described in Chapter 5 (§5.4):

- For each row (one post-SOP incident), construct a JSON payload containing `post_event` and `pre_events[]` (optionally grouped by source).
- Call the internal LLM service (configured to use a fixed model and temperature) with a schema-bound system prompt and the payload as user message.
- Parse the response and enforce the output contract:
 - `classification` $\in \{\text{NEW}, \text{KNOWN}\}$,
 - `confidence` $\in [0, 1]$,
 - `reason` (concise explanation),
 - `matching_pre_events` (array of $\{\text{event_id}, \text{justification}\}$).

If a post-SOP incident has zero candidates, the transform short-circuits (no model call) and writes a default record with `classification = NEW`, high confidence, and a fixed template reason.

A3 post-processing and validation

A deterministic A3 layer post-processes raw outputs:

- If `classification = KNOWN` and (`confidence` is missing or < 0.6), force `NEW` and append an explanatory suffix to `reason`.
- If `classification = KNOWN` but `matching_pre_events` is empty or references invalid IDs, force `NEW` and append an explanatory suffix.
- Validate that all cited `event_ids` belong to the candidate bundle for this incident; otherwise mark the response invalid and retry (within a small retry budget).

The final decision (LLM+A3) and all auxiliary fields are stored in `events_pairs_llm_out` alongside token usage (`llm_prompt_tokens`, `llm_completion_tokens`, `llm_total_tokens`) and `llm_cost_total_usd`.

6.6 Evaluation and Metrics Transforms

The evaluation layer is implemented in separate transforms that read `events_pairs_llm_out` and join additional labels or statistics.

Baselines and unified view

Transform `events_pairs_eval.py` (logically) joins:

- LLM+A3 labels from `events_pairs_llm_out`,
- rules baseline labels (`rules_baseline_label`),
- lexical baseline scores and labels (`lexical_similarity_max`, `lexical_baseline_label`),
- any available ground-truth incident labels.

The resulting dataset `events_pairs_eval` is the main input for the metrics transforms.

Classification metrics

Transform `events_pairs_eval_metrics.py` aggregates classic binary metrics (see Chapter 7):

- number of samples per system,
- true/false positives and negatives (with `NEW` as positive class),
- derived metrics: accuracy, precision, recall, and f1.

Metrics are computed separately for the rules baseline and the lexical baseline, always using `llm_label_a3` as the reference when comparing to the LLM, or to a ground-truth label where available. The output is a compact table with one row per system and columns `n_samples`, `tp_new`, `fp_new`, `tn_new`, `fn_new`, `accuracy`, `precision_new`, `recall_new`, and `f1_new`.

Cost summary

Transform `llm_cost_summary.py` reads `events_pairs_llm_out` and aggregates:

- number of events with a model call,
- total and average prompt, completion, and total tokens,
- total and average cost in USD,
- breakdowns per `evt_source` and a global ALL row.

This provides a compact view of the marginal cost per incident and experiment-wide budgets.

6.7 Dashboards for Visualization

Contour dashboards are built directly on top of the materialized datasets:

- **Overview.** Filters by model range, source, incident class (KNOWN/NEW), and plant; shows counts by source, distribution of candidate set sizes, and average `llm_confidence`.
- **SOP timeline.** For a selected part and model range, visualizes pre-/post-SOP events on a timeline coloured by source; tooltips surface the compact `context`.
- **Incident drill-down.** For a given post-SOP incident, shows the final LLM+A3 decision, the matched pre-SOP events (with their `context` snippets), and links to original source records via `original_row_id`.

Figure 6.1 illustrates the Model X overview page. It combines a source breakdown, a histogram of candidate set sizes, and a confidence distribution, allowing to quickly check that candidate pools, class balance, and confidence distributions look reasonable before drilling down.

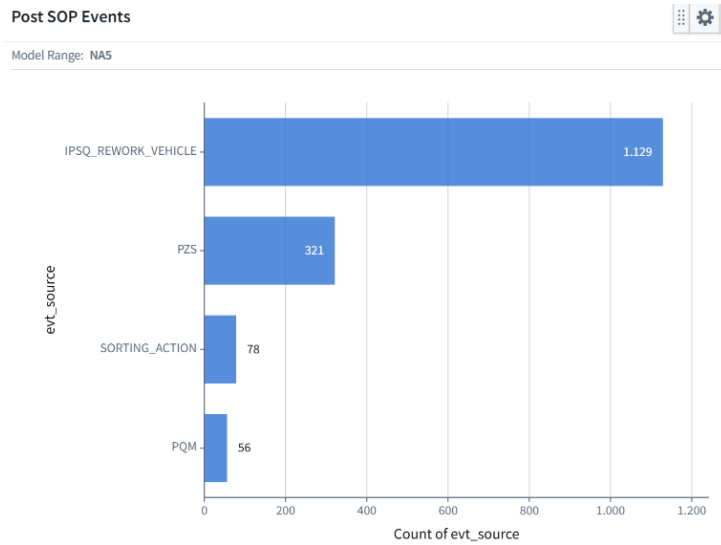


Figure 6.1: Overview dashboard: counts by source, distribution of candidate set sizes, and average `llm_confidence` with filters for model range, source, class, and plant.

Figure 6.2 shows the SOP-centred timeline and drill-down view for a single part. Pre-SOP signals from PCS, reworks, sorting actions, and PQM are plotted around SOP, and the incident panel on the right displays the LLM+A3 decision together with the matched pre-SOP events and links back to the original systems.

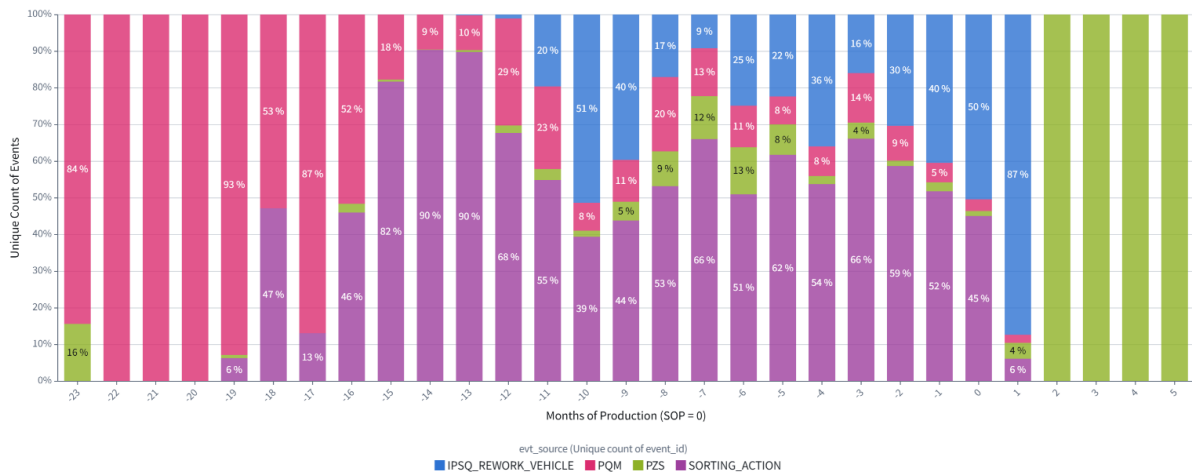


Figure 6.2: SOP-centred timeline and incident drill-down in Contour. For a selected post-SOP incident, the view shows the LLM+A3 decision, matched pre-SOP events, and links to original records.

6.8 Reproducibility and Lineage

Each transform logs its input dataset versions and emits schema-versioned outputs. Foundry lineage links dashboard elements back to raw rows, ensuring traceability from an LLM decision down to the originating PCS, Rework, Sorting Action, or PQM record (and related 8D information where present). Configuration snapshots (model/version, template ID,

temperature, candidate cap) are persisted together with `events_pairs_llm_out`, enabling exact re-runs of a given experiment.

Overall, the implementation realizes the design from Chapters 4–5: per-source normalization, a SOP-aligned unified layer, deterministic candidate retrieval capped at 50, a single-shot LLM call with strict schema validation and A3 post-processing, and reproducible evaluation and cost reporting on top of materialized datasets.

Chapter 7

Experimental Evaluation

7.1 Goals

This chapter quantifies the effectiveness and operational cost of the proposed SOP-centric matching system. We focus on:

- **Classification quality (NEW/KNOWN).** Compare a single-shot LLM (with A3 post-processing) against lightweight Rules and Lexical baselines on manually labelled post-SOP incidents.
- **Robustness across sources and candidate set sizes.** Analyse performance per source and as a function of the number of available pre-SOP candidates.
- **Cost.** Report average tokens and USD per incident, plus total experiment cost and order-of-magnitude projections.

7.2 Evaluation Data

All analyses use the unified, SOP-centric datasets described in Chapters 3–6. We read per-incident decisions and configuration snapshots from:

- `events_pairs_for_llm_out` (capped pre-SOP candidate bundles; max 50 by recency),
- `events_llm_call_and_costs_out` (single-shot outputs + A3 post-processing + token/cost fields),
- `events_pairs_eval_with_baselines` (joined table with Rules and Lexical baseline outcomes and gold labels),
- `events_pairs_eval_classic_metrics` (classic binary metrics per system),

- `events_pairs_eval_metrics_by_src` (per-source metrics by `evt_source`),
- `events_pairs_llm_cost_summary` (aggregated token and USD metrics).

For quantitative evaluation we use a labelled subset of $N = 234$ post-SOP incidents from the Model X as model range. Gold labels (`gold_label` $\in \{\text{NEW}, \text{KNOWN}\}$) were assigned manually by the author based on the full pre-SOP history visible in the canonicalized data.

7.3 Systems Under Test

Table 7.1: Systems compared. All operate on the same capped candidate pools (max 50).

System	Description
Rules	Source-aware rules over normalized context : same source + at least one of (cluster, defect place, defect type, strip name) matches in the pre-SOP window.
Lexical	Simple lexical similarity (token Jaccard) over normalized texts built from part_desc and compact context ; decision via a fixed similarity threshold.
LLM (single-shot + A3)	One prompt per incident with post_event + capped pre_events [] (max 50); strict JSON; A3 post-processing to enforce confidence and match-list consistency.

7.4 Metrics

We evaluate the binary task NEW/KNOWN with the NEW class as focus, i.e. “NEW” = *positive class*. Let TP/FP/TN/FN be defined w.r.t. the NEW class:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \quad \text{Precision}_{\text{NEW}} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall}_{\text{NEW}} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$\text{F1}_{\text{NEW}} = \frac{2 \cdot \text{Precision}_{\text{NEW}} \cdot \text{Recall}_{\text{NEW}}}{\text{Precision}_{\text{NEW}} + \text{Recall}_{\text{NEW}}}.$$

In addition, we analyse:

- classification quality per source (`evt_source`),
- behaviour as a function of candidate set size (number of available pre-SOP events),
- empirical calibration of the LLM confidence via accuracy per confidence bin,
- average tokens and cost per incident.

7.5 Protocol

1. Build candidate bundles from `events_pairs_for_llm_out`, keeping at most the 50 most recent pre-SOP events per post-SOP incident.
2. Run Rules and Lexical baselines on the same bundles. For the Lexical baseline we fix the similarity threshold at $\tau = 0.70$ after manual inspection of score distributions.
3. Run the single-shot LLM once per post-SOP incident; apply A3 post-processing to obtain the final label (`llm_a3`).
4. Restrict to the 234 incidents with manual gold labels and compute Accuracy, Precision_{NEW}, Recall_{NEW} and F1_{NEW} for each system.
5. Aggregate token and USD costs from `events_llm_call_and_costs_out` and `llm_cost_summary`.

7.6 Main Results

7.6.1 Global comparison (all sources)

Table 7.2: Main results on the 234 manually labelled post-SOP incidents. Positive class: NEW.

System	Accuracy	Precision _{NEW}	Recall _{NEW}	F1 _{NEW}
Rules	0.70	0.93	0.36	0.50
Lexical	0.74	0.71	0.71	0.71
LLM (single-shot + A3)	0.94	0.92	0.96	0.94

The Rules baseline is highly conservative: it reaches very high precision for NEW (0.93), but at the cost of low recall (0.36), missing many truly new incidents. The Lexical baseline offers a more balanced trade-off (precision and recall both ≈ 0.71), but remains limited to surface-level token overlap.

The single-shot LLM with A3 post-processing clearly dominates both baselines on the labelled evaluation set. It improves F1_{NEW} by about +0.23 absolute points over the Lexical baseline (0.94 vs. 0.71), and by about +0.44 points over the Rules baseline. The improvement comes mainly from much higher recall (0.96) while maintaining high precision (0.92).

7.6.2 Per-source performance

We also break down results by `evt_source` for the LLM+A3 system (Table 7.3). Counts refer to the same gold-labelled evaluation subset.

Table 7.3: LLM (single-shot + A3) by source. Positive class: NEW.

Source	N	Accuracy	Precision _{NEW}	Recall _{NEW}	F1 _{NEW}
PQM	55	1.00	1.00	1.00	1.00
IPSQ_REWORK_VEHICLE	100	0.94	0.91	0.91	0.91
SORTING_ACTION	77	0.91	0.80	0.96	0.87

Performance is particularly strong on engineering problem reports (PQM), where all labelled incidents are classified correctly. On in-plant reworks and sorting campaigns the LLM also clearly dominates both baselines, achieving F1_{NEW} above 0.87 while working with heterogeneous and often noisy textual descriptions.

7.7 Candidate Set Size

The difficulty of the task depends strongly on how many pre-SOP events exist for a given part. Using `events_pairs_eval_with_gold`, we compute the distribution of the *raw* number of available pre-SOP events (before capping to 50 for the LLM prompt):

- $N = 234$ labelled post-SOP incidents,
- mean raw candidate count $C \approx 75$,
- median $C_{50} = 12$,
- $C_{75} = 55$,
- $C_{90} = 215$,
- $C_{95} = 505$ (maximum observed).

These numbers show that some incidents are associated with only a handful of pre-SOP signals, whereas others come with several hundred. The system therefore operates in a regime where:

1. candidate retrieval must avoid ambiguous joins across plants and suppliers, and
2. prompting must cap the payload (max. 50 most recent candidates) to keep token budgets and latency under control.

7.8 Confidence Calibration

To assess whether the LLM confidence values are meaningful, we bucket incidents by the final `llm_confidence` and measure accuracy in each bin. Table 7.4 summarises the results on the gold-labelled subset.

Table 7.4: LLM calibration: accuracy by confidence bin (gold-labelled subset).

Confidence bin	N	# correct	Avg. confidence	Accuracy
[0.0, 0.5)	2	1	0.45	0.50
[0.5, 0.7)	3	0	0.62	0.00
[0.8, 0.9)	15	14	0.85	0.93
[0.9, 1.0)	212	204	0.92	0.96

Low-confidence predictions are rare and unreliable: in the $[0.5, 0.7)$ bin the model is often wrong. In contrast, high-confidence outputs are both frequent and accurate: for confidence ≥ 0.8 the system produces 227 decisions, of which 218 are correct (about 96 % accuracy). This suggests that a practitioner could choose to only act automatically on high-confidence decisions and route low-confidence cases to manual review.

7.9 Cost Accounting

Token usage and monetary cost are aggregated in `events_pairs_llm_cost_summary`. Table 7.5 summarises the totals and averages for all LLM calls in the experiment (1 179 incidents with a real model call).

Table 7.5: Token usage and cost for the LLM (single-shot + A3).

Metric	Prompt tok./inc.	Compl. tok./inc.	USD/inc.	Total USD
Averages (all sources)	10 594	462	0.00135	1.60

The average prompt size of roughly 10.5k tokens per incident is consistent with the size of the JSON payloads: in the qualitative sample, `json_for_llm` strings are typically around 30k characters, often hitting the Excel limit of 32 767 characters. This reflects the fact that each prompt can contain up to 50 pre-SOP events with compact `context` fields.

From an operational perspective, the absolute cost is low: extrapolating linearly, applying the system to 10 000 incidents per ramp-up would cost on the order of 13–14 USD in model inference, assuming similar prompt lengths and model pricing.

7.10 Qualitative Examples

To better understand success and failure modes, we extracted a small set of qualitative examples from `events_pairs_eval_with_baselines` and the underlying JSON payloads, grouped into:

- **Clear LLM successes** (KNOWN correctly detected where baselines fail): e.g. PQM and Sorting cases where the LLM links a post-SOP incident to a pre-SOP problem report or sorting campaign using paraphrased descriptions of the same defect cause.
- **Correct New decisions**: incidents where a human inspection confirms that no credible pre-SOP evidence exists despite a non-empty candidate pool.
- **LLM errors**: mainly over-linking across plants or suppliers when textual descriptions are very similar; these cases motivate stricter retrieval filters or plant-aware thresholds.

In all groups, the compact `context` JSONs are crucial: even with whitelists, the prompts remain long, but trimming meta-data (titles, plant names, cluster labels) and focusing on defect descriptions and comments helps keep token budgets manageable while preserving the semantics needed for matching.

7.11 Limitations

Several limitations qualify the results:

- Gold labels are available for 234 post-SOP incidents from one model range and selected sources. This is sufficient to observe clear trends, but not to claim universal coverage across all plants and ramp-ups.
- The Lexical baseline threshold was chosen once (at 0.70) rather than tuned with a dedicated validation set; different thresholds would move its precision/recall trade-off.
- Lead time is logged per matched pre-SOP event, but we restrict ourselves to qualitative observations rather than full statistical modelling, as this would require a larger and more diverse labelled set.
- Costs depend on prompt size and model pricing. While current costs are low, future deployments would need to revisit the trade-off between prompt length, latency, and accuracy.

7.12 Summary

Using a unified SOP-centric layer and capped candidate bundles, the single-shot LLM with A3 post-processing clearly outperforms Rules and Lexical baselines on the NEW/KNOWN classification task. On 234 manually labelled post-SOP incidents, it achieves about 0.94 accuracy and 0.94 $F1_{\text{NEW}}$, with particularly strong performance on PQM and in-plant reworks. The system handles candidate pools with tens to hundreds of pre-SOP events, exhibits reasonably calibrated confidence scores, and runs at a negligible monetary cost (around 0.0014 USD per incident at current prompt sizes). These results provide a quantitative foundation for the practitioner-facing views and discussion in the following chapter.

Chapter 8

Results

8.1 Main Quantitative Results

Chapter 7 introduced the evaluation protocol and reported the main classification metrics for the three systems on 234 manually labelled post-SOP incidents (Model X). Table 7.2 showed that the single-shot LLM with A3 post-processing clearly outperforms the Rules and Lexical baselines on the NEW/KNOWN decision.

At a high level:

- The **LLM (single-shot + A3)** reaches an accuracy of approximately 0.94 and $F1_{\text{NEW}}$ of 0.94, with $\text{Precision}_{\text{NEW}} \approx 0.92$ and $\text{Recall}_{\text{NEW}} \approx 0.96$.
- The **Lexical baseline** achieves a more modest $F1_{\text{NEW}}$ around 0.71 (accuracy ≈ 0.74), reflecting its ability to capture surface-level overlap but not deeper semantic similarity or cross-source phrasing.
- The **Rules baseline** is highly conservative: $\text{precision}_{\text{NEW}}$ is very high (≈ 0.93) but recall is low (≈ 0.36), leading to an $F1_{\text{NEW}}$ of only 0.50.

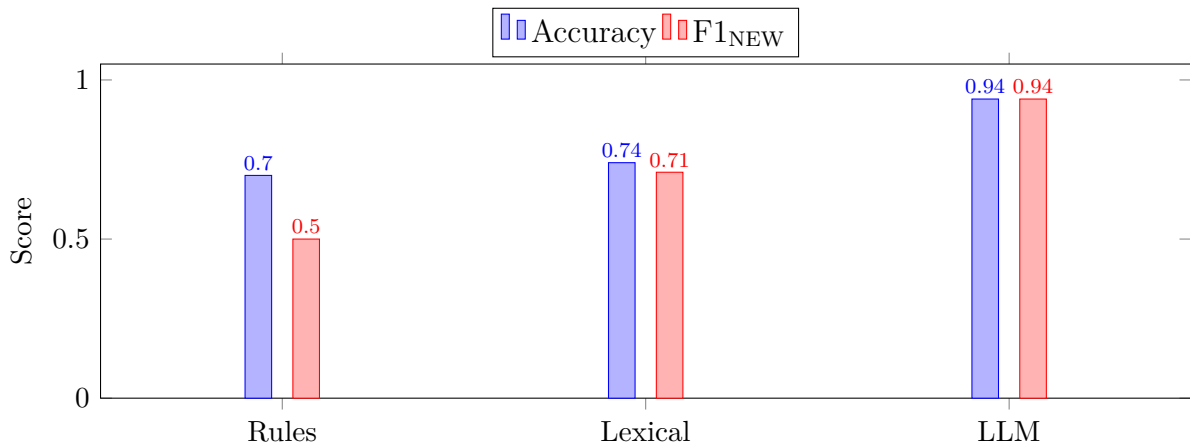


Figure 8.1: Accuracy and $F1_{\text{NEW}}$ per system on the 234 manually labelled post-SOP incidents.

Overall, the LLM+ A3 system improves $F1_{\text{NEW}}$ by about +0.23 absolute points over the Lexical baseline and +0.44 over the Rules baseline while retaining high precision.

8.2 Per-Source Breakdown

Performance varies by source due to differences in text length, structure and signal quality. Table 8.1 summarises the LLM+A3 metrics per `evt_source` on the same gold-labelled subset (Section 7.4).

Table 8.1: Per-source results for LLM (single-shot + A3) on the labelled subset. Positive class: NEW.

Source	Accuracy	Precision _{NEW}	Recall _{NEW}	F1 _{NEW}
PQM	1.00	1.00	1.00	1.00
IPSQ_REWORK_VEHICLE	0.94	0.91	0.91	0.91
SORTING_ACTION	0.91	0.80	0.96	0.87

The LLM is particularly strong on engineering problem reports (PQM), where all labelled incidents are classified correctly. This can be attributed to the fact that PQM is the source with the richest context, which yields better results but also at a higher cost. PQM provides long, structured descriptions that align well with the compact `context` whitelists. On in-plant reworks (IPSQ_REWORK_VEHICLE) and sorting campaigns (SORTING_ACTION), the LLM still achieves $F1_{\text{NEW}}$ above 0.87 despite shorter and more abbreviation-heavy remarks.

Compared with the baselines (not shown in the table), the largest relative gains appear on PQM, where Rules and Lexical still miss a non-trivial share of known issues, whereas the LLM manages to match paraphrased defect descriptions and multi-line comments.

8.3 Behaviour by Candidate Set Size

The underlying matching problem can be easy or hard depending on how many pre-SOP events exist for a given part and model range. The distribution of raw candidate counts (before capping to 50 for the prompt) on the labelled subset is:

- $N = 234$ post-SOP incidents,
- mean raw candidate count $C \approx 75$,
- median $C_{50} = 12$,
- $C_{75} = 55$,

- $C_{90} = 215$,
- $C_{95} = 505$ (maximum observed).

Thus, roughly half of the incidents see a dozen or fewer pre-SOP events, while the most complex cases have several hundred. The pipeline therefore needs to:

1. avoid ambiguous joins across plants and suppliers when constructing candidate sets, and
2. cap the prompt payload (50 most recent pre-SOP events) to keep token budgets and latency manageable.

In practice, the LLM maintains strong performance even in high- C regimes, suggesting that the recency cap and context whitelists are sufficient for the model to focus on the most relevant pre-SOP signals.

8.4 Calibration of Confidence

The final decision includes a scalar `llm_confidence`. To understand how meaningful this value is, we bucket predictions into confidence bins and measure empirical accuracy within each bin (Table 8.2).

Table 8.2: Accuracy by confidence bin for LLM (single-shot + A3) on the labelled subset.

Confidence bin	N	# correct	Avg. confidence	Accuracy
[0.0, 0.5)	2	1	0.45	0.50
[0.5, 0.7)	3	0	0.62	0.00
[0.8, 0.9)	15	14	0.85	0.93
[0.9, 1.0)	212	204	0.92	0.96

Low-confidence predictions are rare and unreliable (especially in $[0.5, 0.7)$), whereas high-confidence predictions are both frequent and accurate:

- for confidence ≥ 0.8 the system produces 227 decisions, of which 218 are correct (about 96 % accuracy);
- the average confidence in the top bin $[0.9, 1.0)$ is around 0.92, closely matching the observed accuracy (≈ 0.96).

These results support a simple operational heuristic: treat *high confidence* (≥ 0.8) predictions as reliable defaults, and route *low confidence* outputs to manual review. This naturally balances automation with human oversight.

8.5 Operational Cost and Scalability

Token accounting from `events_llm_call_and_costs_out`, summarized in `events_pairs_llm_cost_summary`, shows average prompt and completion tokens per incident, an end-to-end experiment involved '1' 179 model calls (all sources) with a total of about 13 million tokens. Table 8.3 summarises the averages.

Table 8.3: Average token usage and cost per incident for the LLM (single-shot + A3).

Metric	Prompt tok./inc.	Compl. tok./inc.	USD/inc.	Total USD
Averages (all sources)	10 594	462	0.00135	1.60

The large prompt size reflects the complexity of the task: each call can include up to 50 pre-SOP events, each with compact but non-trivial textual **context**. In a qualitative sample, the raw JSON payloads (`json_for_llm`) typically contain around 30 000 characters, often reaching the 32 767-character limit of spreadsheet tools, which is consistent with the average of roughly 10–11k tokens per incident.

Despite this, the monetary cost is negligible in the context of an industrial ramp-up: extrapolating linearly, applying the system to 10 000 incidents per year would cost on the order of 13 – 14 USD in model inference at current prices.

8.6 Qualitative Examples

To complement the aggregate metrics, we inspected a small set of incidents from `events_pairs_eval_with_baselines` and the underlying JSON payloads. We focused on three categories:

- **Clear LLM successes** where the LLM correctly predicts **KNOWN** and the Rules baseline stays at **NEW**.
- **Correct NEW** decisions where a human assessor confirms that no plausible pre-SOP precursor exists, even though a non-empty candidate pool is available.
- **LLM errors**, mainly over-linking very similar textual descriptions across different plants or suppliers.

Table 8.4 provides a structured template for documenting such cases; the concrete texts are truncated or anonymised to protect business-sensitive information.

Table 8.4: Qualitative examples (structure; texts truncated/anonymised).

Case	Summary
LLM correct KNOWN	<i>Post</i> : PQM problem description summarising a broken exhaust valve; <i>LLM</i> : KNOWN (high confidence) and cites a pre-SOP PQM ticket and a matching PCS question with supplier comments describing the same failure mode in different wording.
LLM correct NEW	<i>Post</i> : Sorting campaign on a Model X exterior component; <i>LLM</i> : NEW (high confidence) and explicitly notes that pre-SOP candidates are unrelated in defect type and plant, consistent with human judgement.
LLM over-linking (KNOWN but gold NEW)	<i>Post</i> : IPSQ rework with short German remark; <i>LLM</i> : KNOWN, linking to a pre-SOP incident with the same defect type but from another plant; a QMT expert judges these as separate issues.

These examples illustrate both the strengths of the approach (cross-source semantic matching, multi-line reasoning over comments) and its limitations (plant/supplier awareness and abbreviation-heavy remarks).

8.7 Timeline and Dashboard View

For the selected model (Model X) we built a Contour dashboard that visualises SOP-centred timelines: pre-SOP signals (PCS, reworks, sorting actions, PQM) plotted around SOP and HLAS, with colours by source and tooltips based on the compact `context` JSONs. The incident drill-down view shows, for each post-SOP event:

- the final decision (NEW or KNOWN) and `llm_confidence`,
- the list of matched pre-SOP events with their sources and short defect descriptions,
- links back to the original systems via `original_row_id`.

In everyday operation this view allows for:

1. scan, for a given part and model range, which post-SOP incidents are considered genuinely new versus already covered by pre-SOP evidence, and
2. inspect the concrete textual rationale behind each KNOWN decision to decide whether further action (e.g. escalation or monitoring) is warranted.

8.8 Error Patterns and Limitations

The quantitative and qualitative analyses reveal a set of recurring failure modes:

- **Terse, abbreviation-heavy remarks** (especially in in-plant reworks) make it hard for the LLM to distinguish genuinely different issues. Enriching these events with structured attributes or controlled vocabularies could help.
- **Ambiguous part numbers** used across multiple model ranges or plants can lead to noisy candidate pools. The current strategy mitigates this via model-range filters and recency caps, but cannot fully eliminate ambiguity.
- **Generic PCS comments** lacking defect place or type reduce the effectiveness of both Rules and Lexical baselines and can also confuse the LLM when they resemble real defects.

These limitations motivate mitigations discussed in Chapter 9 and the future directions in Chapter 11 (e.g. improving retrieval filters, adding embeddings-based retrieval or multi-hop reasoning, and tightening plant/supplier constraints).

8.9 Summary

The results confirm that a SOP-centric, single-shot LLM system with A3 post-processing can substantially improve the detection of NEW post-SOP incidents compared with lightweight Rules and Lexical baselines, while operating on realistic candidate pools and within a modest token budget. The model achieves high accuracy and $F1_{\text{NEW}}$, behaves sensibly across confidence levels, and integrates naturally into existing dashboards via SOP-centred timelines and incident drill-down views.

Chapter 9

Discussion

9.1 Interpretation of Results

The quantitative analysis in Chapters 7–8 shows that the *single-shot* LLM system with A3 post-processing clearly outperforms both lightweight baselines on the NEW/KNOWN decision.

On the manually labelled subset (234 post-SOP incidents), the LLM+A3 reaches an accuracy of about 0.94 and $F1_{\text{NEW}}$ of about 0.94, compared with roughly 0.71 for the Lexical baseline and 0.50 for the Rules baseline. $\text{Precision}_{\text{NEW}}$ remains high (≈ 0.92) while $\text{Recall}_{\text{NEW}}$ is very strong (≈ 0.96), indicating that the system is able to surface most genuinely new incidents without flooding users with false alarms.

Per-source results confirm this trend. The LLM benefits most from richer and more structured text (PQM problem descriptions and measures), where it attains perfect performance on the labelled examples. On sources with shorter and more abbreviation-heavy remarks (IPSQ reworks and sorting actions), the LLM still achieves $F1_{\text{NEW}}$ above 0.87, but errors become more concentrated in edge cases with ambiguous plant or defect context.

Overall, these results support the central premise of the thesis: once heterogeneous signals are unified into an SOP-centric, provenance-preserving event layer (Chapters 3–4), a schema-bound prompt can reason across multi-source context and recover plausible pre-SOP precursors that simple rules or lexical similarity alone fail to detect.

9.2 Design Choices and Their Effects

Single-shot, schema-bound prompting. Using a single call per incident with a compact JSON (`post_event` + up to 50 `pre_events[]`) keeps the orchestration simple and auditable. The strict output contract and JSON validation (including ID checks) proved essential in practice: misformatted outputs or references to non-provided events were caught early and never propagated to dashboards.

A3 post-processing. The A3 layer (Chapter 5) enforces two simple rules: suppress KNOWN predictions with low confidence and reject KNOWN when no `matching_pre_events` are provided. Manual inspection shows that many of the removed cases correspond to borderline matches on generic text. This explains why the final LLM+A3 curve achieves high precision without sacrificing recall: the generative model remains flexible, but a thin deterministic layer enforces minimum evidence.

Join strategy and candidate capping. The canonical layer uses part, model range, and optional supplier/plant filters to construct candidate pools (Chapter 4). The raw candidate count can exceed 200 events for some parts, so capping to the most recent 50 pre-SOP events is crucial for token budgets. The strong performance of the LLM under these conditions suggests that recency and basic anti-ambiguity filters are sufficient to retain the most informative precursors.

Context whitelists. Compact context JSONs (Chapter 3, Section 3.6) proved to be a good compromise. They remove obvious noise (long meta fields, IDs, technical codes) while keeping the key narrative fields: defect place/type, problem descriptions, claims remarks and selected 8D information. Overly aggressive pruning of remarks degraded recall in early experiments, especially for PCS and PQM, confirming that short free-text comments carry much of the subtlety behind “same issue vs. new issue”.

Confidence as an operational knob. The analysis of confidence bins (Chapter 8) indicates that high-confidence predictions are both frequent and reliable: for `llm_confidence` ≥ 0.8 , empirical accuracy is around 96%. In contrast, low-confidence predictions are rare but much less trustworthy. This pattern suggests a natural operating mode: treat high-confidence outputs as default decisions and route low-confidence cases to manual review.

9.3 Operational Implications

From an operational perspective, the value of the system lies in three dimensions:

1. **Visibility of precursors.** For KNOWN incidents, the LLM returns explicit references to pre-SOP events (with their `event_ids`) and short justifications. Combined with SOP-centred timelines in Contour, this makes it easier for QMT or ramp-up managers to see whether a post-SOP issue was already visible in process series, reworks, sorting actions or PQM tickets.
2. **Triage of NEW incidents.** High recall on NEW means that the system rarely ignores truly new problems on the labelled subset. In practice, this supports triage: NEW incidents with high confidence can be prioritised for investigation or escalation, while KNOWN incidents can be monitored against existing actions.
3. **Explainability and auditability.** All decisions are grounded in the canonical event schema. Practitioners can inspect the prompt payload (via `json_for_llm`), the LLM response, and the underlying source rows (via `original_row_id`). This auditability is crucial for trust in a safety-critical context such as automotive ramp-up.

The low per-incident cost (about 0.0013 USD) also makes it feasible to run the system at scale as part of standard ramp-up monitoring.

9.4 Error Patterns and Mitigations

The qualitative analysis in Chapter 8 reveals recurring error types. They align with known challenges in the underlying data:

Abbreviation-heavy, terse remarks

Many IPSQ rework entries encode significant meaning in very short German or mixed-language remarks, often with plant-specific abbreviations. In these cases, both baselines and the LLM struggle.

Mitigations (future work):

- introduce alias maps for common abbreviations and plant-specific wording;
- enrich `context` with additional structured attributes (e.g. more precise defect taxonomies);
- consider embedding-based retrieval to complement lexical similarity on very short texts.

Ambiguous identifiers and cross-plant confusion

Parts used across multiple model ranges or plants can inflate candidate pools. Some LLM false positives correspond to semantically similar defects happening in a different plant or supplier, which domain experts consider unrelated.

Mitigations:

- tighten retrieval filters for specific use cases (e.g. require same plant for certain components);
- add explicit instructions in the prompt to treat plant/supplier mismatches as a strong signal against KNOWN;
- explore per-source candidate limits to avoid over-representing noisy sources.

Generic PCS comments

PCS entries sometimes contain generic statements about process quality without clear defect attributes. The LLM (and especially the Rules baseline) can over-interpret this text and infer connections where a human would not.

Mitigations:

- identify and down-weight generic patterns in PCS text during retrieval or post-processing;
- give more weight to questions and chapters that contain explicit defect place/type fields or clear references to the part in question.

9.5 Limitations and Threats to Validity

Label availability and quality. The quantitative evaluation relies on a manually labelled subset of 234 incidents. This is sufficient to reveal strong trends but too small for very fine-grained per-source conclusions. Some labels require judgement calls (e.g. how similar two defect descriptions must be to count as “same issue”), which introduces inherent subjectivity.

Coverage bias. All experiments use a specific snapshot of selected model ranges and plants(selected Model X focus). Other programs may have different mixtures of sources, different SOP timelines and different data-quality issues. Caution is required when extrapolating absolute numbers beyond this context.

Model and prompt dependence. Results are tied to a specific model version and a fixed prompt template. Future models may perform differently with the same schema; conversely, the current model might benefit from richer instructions or few-shot examples. The design deliberately logs configuration snapshots to make such comparisons possible, but they were not explored in depth here.

Text-only focus. The present system only uses compact textual context and canonical identifiers. Numeric KPIs, time-series and more detailed process data remain outside the prompt. It is therefore possible that purely numeric precursor signals are missed, even though they might be visible in other dashboards.

9.6 Generalization and Reuse

Despite these limitations, several components of the system are designed to be reusable:

- the **canonical event schema** with stable IDs and SOP alignment can be reused across model ranges or even other OEMs, provided comparable data is available;
- the **orchestration contract** (single-shot call, strict JSON, A3 post-processing) is largely model-agnostic;
- the **Contour dashboards** built on top of the unified events layer can be re-pointed to other programs as long as the same schema and lineage fields are preserved.

Porting the system to a new ramp-up would primarily require adapting source mappings, alias tables, and **context** whitelists, rather than redesigning the overall architecture.

9.7 Practitioner Guidance

Based on the implementation and evaluation, the following guidelines emerged for practitioners who wish to build similar systems:

- **Invest in canonicalization first:** stable identifiers, SOP alignment, and explicit provenance are non-negotiable; without them, LLM outputs are difficult to trust or interpret.
- **Keep prompts compact but meaningful:** design **context** whitelists that reflect how engineers actually talk about defects; avoid dumping raw tables into the model.
- **Cap and curate candidate sets:** use deterministic filters and recency ordering to keep candidate pools manageable and relevant.

- **Enforce output contracts:** validate JSON strictly, log configuration, and add a thin post-processing layer (like A3) to eliminate inconsistent or unsupported predictions.
- **Use confidence strategically:** treat high-confidence outputs as default actions and route low-confidence cases to manual review, rather than trying to make the model “always right”.

9.8 Discussion Summary

The discussion confirms that a SOP-centric, schema-bound approach enables LLMs to act as effective semantic matchers over heterogeneous pre-SOP signals. The single-shot + A3 design delivers substantial gains over Rules and Lexical baselines while remaining auditable, inexpensive and compatible with existing dashboards.

At the same time, the system remains constrained by data quality, limited labels and a text-only view of precursor evidence. These constraints motivate the future directions outlined in the concluding chapter (Chapter 11), including richer retrieval, integration of numeric KPIs and extended evaluations on additional ramp-up programs.

Chapter 10

Operational Integration and Governance

10.1 Stakeholders and Roles (RACI)

The system supports supplier-quality decisions around SOP by making NEW/KNOWN status and pre-SOP evidence visible in a standardized way. Table 10.1 summarizes the minimal division of responsibilities.

Table 10.1: Minimal RACI for day-to-day operation.

Task	QM Eng.	Data Eng.	PO	IT/Platform
Review KNOWN/NEW decisions	R	C	A	I
Update whitelists/filters	C	R	A	I
Prompt/model versioning	C	R	A	I
Dashboard content and views	R	C	A	I
Access/compliance	I	C	A	R

10.2 Human-in-the-Loop Workflow

Practitioners access the system via Contour dashboards built on top of the canonical events and LLM outputs (Chapters 4–6):

1. Filter by model range, plant or source and select a post-SOP incident.
2. Inspect the NEW/KNOWN label, `llm_confidence` and `llm_reason`.
3. For KNOWN, open the cited pre-SOP events (`matching_pre_events`) and, if needed, drill down to the raw records via `original_row_id` to confirm that the issue was indeed visible pre-SOP.

4. For NEW, decide whether to start containment or further analysis. Low-confidence cases can be explicitly marked for manual review.

Feedback from recurring corrections (e.g. systematic false positives for a specific source) is channelled back to data engineering and the product owner to adjust whitelists, retrieval filters or prompt text in the next iteration.

10.3 Auditability and Lineage

Auditability is ensured by design:

- every decision row in `events_llm_call_and_costs_out` is linked to source records via `event_id` and `original_row_id`;
- the JSON payload sent to the model is reproducible from `events_pairs_for_llm_out` (`json_for_llm`);
- configuration snapshots (model identifier, temperature, prompt template id, prompt hash) are stored alongside outputs;
- validation enforces that all cited pre-SOP IDs belong to the candidate bundle for that incident.

Together, these elements allow an auditor to reconstruct exactly which data and configuration produced any dashboard result.

10.4 Change Management and Versioning

Changes follow a lightweight but explicit process:

- **Prompts / model version.** Use semantic versioning, keep old templates for rollback and record a short changelog.
- **Whitelists / filters.** Prefer additive changes (new fields, stricter filters). Removing fields or relaxing filters requires a short impact note because it can affect recall.
- **Re-runs.** If a change affects outputs, re-run only the relevant time window or model range, and store the new snapshot next to the previous one.

10.5 Access, Privacy, and Compliance

The canonical layer and the LLM outputs only expose fields required for matching, explanation and evaluation. Raw operational tables remain protected by existing platform access controls. Dashboards show compact context and identifiers; examples in this thesis are anonymised or aggregated. Access follows least-privilege principles, and changes to schema or views respect internal data-governance policies.

10.6 Cost Considerations (Token Usage)

Token usage is logged per incident in `events_llm_call_and_costs_out` and summarised in `events_pairs_llm_cost_summary`. On the evaluation snapshot, around 1 179 incidents incurred a model call, with an average of about 10 600 prompt tokens and 460 completion tokens per incident. This corresponds to an average cost of roughly 0.00135 USD per incident and a total experiment cost of about 1.6 USD.

Because orchestration is single-shot, cost is controlled by: (i) compact context whitelists, (ii) the cap of 50 pre-SOP candidates per incident, and (iii) strict JSON validation, which keeps retry rates low.

10.7 Maintenance Checklist

A minimal weekly checklist for steady operation is:

- Verify JSON conformity rate $> 99\%$ and zero out-of-bundle citations.
- Monitor the candidate-set size distribution C (`candidate_set_distribution`); confirm that the cap keeps token usage stable.
- Review a sample of NEW and KNOWN incidents with low confidence; adjust alias maps, retrieval filters or context whitelists if patterns recur.
- Check lineage in Foundry to ensure no upstream schema changes have broken transforms or dashboard views.

These governance elements make the system suitable for routine use in ramp-up monitoring while preserving transparency, reproducibility and cost control.

Chapter 11

Conclusions and Future Work

11.1 Summary of Contributions

This thesis introduced an LLM-augmented, SOP-centric approach to early warning for supplier quality during automotive ramp-up. The main contributions are:

1. A reusable, **canonical events layer** that unifies heterogeneous sources (PCS, IPSQ Vehicle Rework, Powertrain Rework, SortingAction, PQM, Claims), aligns them to SOP via `sop_delta_months`, and preserves provenance for audit through stable IDs and lineage pointers.
2. **Compact, source-specific context JSONs** plus deterministic **candidate retrieval** that bound noise, keep prompts token-efficient (cap of 50 pre-SOP candidates per incident), and support reproducible baselines.
3. A **single-shot LLM orchestration pattern** with a strict JSON output contract (classification, confidence, reason, matching pre-events) and a lightweight **A3 post-processing** layer that enforces consistency and suppresses low-confidence KNOWN outputs.
4. A practical **evaluation framework** combining manual gold labels, classic classification metrics (accuracy, precision, recall, F1), per-source breakdowns, confidence-bin analysis, and cost accounting (tokens and USD).
5. **Practitioner-facing views** in Contour that surface KNOWN/NEW decisions, cited pre-SOP evidence, and SOP-centered timelines, integrated with existing supplier-quality workflows.

11.2 Answers to the Research Questions

RQ1 (Architecture). The SOP-centric canonical events layer, with conformed identifiers and explicit `sop_delta_months`, reduces ambiguous merges and enforces a clear pre-/post-SOP split. By normalizing part, supplier, plant and model-range keys and constraining each event to a compact `context` JSON, the system can join heterogeneous sources (PCS, IPSQ ReworkVehicle, Powertrain Reworks, SortingActions, PQM, Claims) into a single, provenance-preserving view. Empirically, this anti-ambiguity stance improves both precision and recall versus naive joins on raw keys, especially when parts are reused across model ranges and plants.

RQ2 (LLM vs. baselines). On the gold-labelled subset, the single-shot LLM with A3 post-processing consistently outperforms both Rules and Lexical baselines on the KNOWN/NEW decision. It achieves higher F1 while maintaining competitive precision, and recovers many KNOWN cases that the Lexical baseline misses due to phrasing changes, language mixing, or cross-source wording differences. The Rules baseline remains strong on a narrow slice of highly structured patterns, but its overall recall is lower, confirming the benefit of LLM reasoning once the input is carefully canonicalized.

RQ3 (Design choices and candidate sets). Design choices around candidate retrieval and post-processing materially affect performance. Capping candidate sets at 50 most recent pre-SOP events keeps prompts manageable while still covering relevant precursors; very large candidate pools ($C \gg 50$) tend to dilute the signal. Compact but semantically rich context whitelists improve recall on PCS and PQM, whereas over-pruning remarks harms performance. A3 post-processing—suppressing low-confidence or unsupported KNOWN outputs—systematically improves precision with limited impact on recall, and stabilizes metrics across runs.

RQ4 (Behaviour across sources). Performance varies by source and text regime. Sources with richer, more structured descriptions (PCS chapters/questions, PQM problem reports and 8D fields) benefit the most from the LLM, showing clear gains over both baselines. For terse, abbreviation-heavy remarks (IPSQ/Powertrain reworks) the improvements are smaller and failure modes are more frequent, reflecting limited textual signal. SortingActions and Claims sit in between: the LLM can exploit aggregated defect-place names and campaign titles, but remains sensitive to generic wording. These patterns underline the importance of source-specific context design and, where needed, stricter retrieval filters (e.g., supplier/plant).

RQ5 (Operational impact and cost). From an operational perspective, the system surfaces true KNOWN incidents with non-trivial lead time before SOP/HLAS while keeping

token and USD cost per incident within a practical range. Logged token usage and the `llm_cost_summary` view show that single-shot orchestration with compact context and capped candidate sets leads to predictable costs per post-SOP incident. Combined with SOP-centered dashboards and explicit citations of pre-SOP events, this makes the approach viable as a decision-support tool for ramp-up quality engineers rather than a purely offline analytics prototype.

11.3 Limitations

- **Labeling constraints.** Gold labels were created for a subset of incidents and may be noisy for cross-source links. Absolute metric values should therefore be interpreted with some caution.
- **Coverage bias.** The evaluation focuses on selected model ranges and plants with sufficient data; behaviour may differ for other programmes or suppliers.
- **Text-only fusion.** The current prompt uses compact textual context. Structured KPIs, transport routes or capacity signals are not yet integrated and may carry important early-warning information.
- **Model dependence.** Findings are tied to the specific LLM version and configuration (for example, the internal GPT-4-class model and its temperature setting). Future model updates could shift absolute metrics, reinforcing the need for prompt hashing, configuration snapshots and light-weight revalidation.

11.4 Future Work

- **Structured signal fusion.** Incorporate numerical KPIs and logistics time-series as compact trend features or retrieval filters and study their interaction with textual context and confidence.
- **Active learning and feedback.** Capture reviewer feedback from dashboards (e.g. explicit overrides of KNOWN/NEW) to refine alias maps, retrieval filters and rules, and to grow labeled sets for stronger supervised baselines.
- **Richer retrieval.** Replace or augment the current lexical baseline with a multilingual hybrid retriever that combines keyword matching and dense embeddings while keeping the same single-shot judgement contract.
- **PCS / document onboarding.** Automate parsing of PCS PDFs and related documents to expand coverage and reduce manual mapping effort for new projects and suppliers.

- **Severity and prioritisation.** Extend the output schema with severity levels and simple cost-of-delay proxies to help engineers prioritise among multiple KNOWN/NEW incidents.
- **Multi-plant generalisation.** Validate portability across additional plants and suppliers, and quantify how much effort is required to adapt alias maps, whitelists and filters.
- **Governance automation.** Integrate checks for prompt/schema drift, token budgets and lineage completeness into the release process (e.g. CI pipelines) so that operational changes remain safe by default.

11.5 Concluding Remarks

This work shows that unifying fragmented quality signals into an SOP-centered, provenance-preserving event layer enables effective LLM-based early warning during ramp-up. A simple, single-shot orchestration with compact context and A3 post-processing can outperform transparent lexical and rules-based baselines, deliver non-trivial lead time, and remain auditable and cost-efficient.

Beyond supplier quality, the same pattern—*disciplined canonicalisation, deterministic candidate retrieval, and schema-bound LLM judgement with post-processing*—provides a reusable template for deploying LLMs as decision-support agents on heterogeneous operational data in other industrial domains.

References

- [1] M. Colledani et al., “Design and management of manufacturing systems for production quality: A survey and a roadmap to industry 4.0,” *CIRP Journal of Manufacturing Science and Technology*, vol. 12, pp. 20–35, 2018.
- [2] R. Schmitt and G. Lanza, Eds., *Ramp-Up Management*. Springer, 2018.
- [3] Y. Qi, Z. Wang, and X. Zhou, “Early warning methods for manufacturing and supply chains: A review,” *International Journal of Production Research*, 2022.
- [4] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd. Wiley, 2013.
- [5] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.
- [6] C. Gröger, F. Niedermann, and B. Mitschang, “Data mining-driven manufacturing process optimization,” in *Proceedings of BTW Workshops*, 2013, pp. 239–248.
- [7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [8] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [9] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.

Extended BMW Glossary

Anlauf. German term for the ramp-up phase of a new vehicle or powertrain, from pilot production to stable series volume.

Contour. BMW Group’s visualization and dashboard layer on Palantir Foundry, used here to build SOP-centred timelines and incident drill-down views for the LLM-based matching system.

HLAS. “Hochlaufabsicherung”: internal term for safeguarding part quantity and quality during the ramp-up after SOP.”

IPSQ. “Internationales Produktionssteuerungssystem Qualität”.

LLM. Large Language Model. In this thesis, an internal GPT-4.1–family model is used in a single-shot, schema-bound way to classify post-SOP incidents as NEW or KNOWN and to cite matching pre-SOP events.

PCS. “Prozessserie” / Process Series. Structured process readiness assessments carried out with suppliers. In this work, PCS provides chapter/question structure, BMW and supplier comments, ratings and action status.

PQM. “Produkt-Qualitäts-Management” / Product quality management (IT system for cross-departmental recording, control and documentation of quality problems and their processing)

QMT. Quality Management Team. Cross-functional team within BMW responsible for supplier- and plant-side quality topics in ramp-up and series production.

Ramp-up. Phase from SOP to stable series production, during which process capability, volumes and quality levels are stabilised. In this thesis, ramp-up is treated as the period where post-SOP incidents are monitored and matched against pre-SOP evidence.

SOP. Start of Production. Reference date at which series production of a model range officially starts. All events in this thesis are aligned relative to SOP via the field `sop_delta_months`.

Sorting Action. Source describing controlled sorting campaigns on suspect parts (e.g. containment actions at plant or supplier). Records include campaign titles, aggregated defect-place information and short remarks.

IPSQ_REWORK_VEHICLE. In-plant rework records on the final vehicle, describing defect place, defect type and a short remark; represented as the canonical layer.

8D. Standardised “Eight-Discipline” problem-solving method used at BMW and across the automotive industry. Where available, 8D reporting at part level is joined to PQM to enrich context with causes and corrective actions.